

回転機構のレーザー受光と楽器間無線 通信による同期・輪唱システム

情報工学科 2 年

24G1059

佐藤 涼菜

—— チーム 40 ——

宇井 祐真 (24G1021) 梅野 大誠 (24G1026)

山口 侑希 (24G1136) 薄井 悠希 (25G1020)

菊池 侑人 (25G1041)

2026 年 7 月 11 日

1 はじめに

1.1 社会的背景・課題・本稿の目的

近年、動画配信サイトやストリーミングサービスの普及によって、誰もが手軽に音楽コンテンツを視聴できる環境が整っている。特に日本では、ゲームやアニメなどのサブカルチャーを起点とした音楽コンテンツが数多く生み出されており、独自の文化として定着している。一方で、このようなデジタル配信の消費形態は、実際に会場へ足を運んで演奏を体感するライブイベントやコンサートとの相乗効果を生み、市場規模の拡大にも寄与している [1][2]。たとえば、博報堂の谷口による分析では、ストリーミングサービスの利用者がサービス内で新たに好みのアーティストを複数発掘し、そのアーティストたちを一度に鑑賞できるライブやフェスへ参加するという行動の流れが形成されつつあり、さらにライブでの体験を通じて別のアーティストと出会うことで、再びストリーミングでの視聴に戻るという音楽消費サイクルが生まれていると指摘されている [4]。すなわち、デジタル配信とライブイベントは互いの顧客を送客し合う関係にあり、この循環が音楽市場全体の拡大を後押ししていると考えられる。実際に、一般社団法人コンサートプロモーターズ協会が実施したライブ市場調査によれば、2025 年におけるライブ市場の年間売上高は 6443 億円、入場者数は 5999 万人にのぼり、デジタル配信の普及以降もライブ市場は堅調に拡大を続けている [3]。図 1 に示すように、国内のライブ・コンサート市場における年間売上額は 2010 年の 1280 億円から 2019 年の 3665 億円まで、10 年間でおよそ 2.9 倍に拡大している。2020 年には新型コロナウイルス感染症の影響により、会場に人を集めること自体が困難となった結果、年間売上額は 779 億円まで急落した。しかし、行動制限の緩和とともに市場は急速に回復し、2022 年には 3984 億円、2023 年には 5140 億円、そして 2025 年には過去最高となる 6443 億円を記録している。この推移から、ライブ会場に足を運ぶという体験に対する需要は、一時的な外的要因によって落ち込むことはあっても、長期的には一貫して拡大する傾向にあると言える。

デジタル配信が普及していく中で、なぜデジタル配信では代替できない価値がライブ会場に存在するのだろうか。その理由の一つとして、ライブ演出に用いられる音響や照明、ライブ参加者自身が感じる振動といった複数の物理的刺激が挙げられる。これらの刺激は、観客一人ひとりに個別に届けられるのではなく、同じ空間にいる観客全員に同時に共有されるという性質を持つ。その結果、観客同士が同じ体験を共有しているという感覚、すなわち空間全体としての一体感が形成される。換言すれば、ライブ会場における没入感とは、ユーザが音楽を聴くだけでとどまらない、物理的、視覚的な技術と同じ空間で同時に受け取ることによって生まれる体験だと考えられる。この仮説は、複数の研究によっても裏付けられている。ライブ演奏とデジタル配信を直接比較した実証研究によっても裏付けられている。Kreuzer ら (2025) は、同一のクラシック音楽コンサートを、会場で聴取する条件と映像配信で視聴する条件とで比較する実験を行った。その結果、没入感や社会的つながりの感覚を含む複数の評価軸において、ライブ条件の方が配信条件よりも有意に高い評価を得ることを示した [5]。すなわち、同じ演奏内容・同じ音質であっても、同じ空間を他の観客

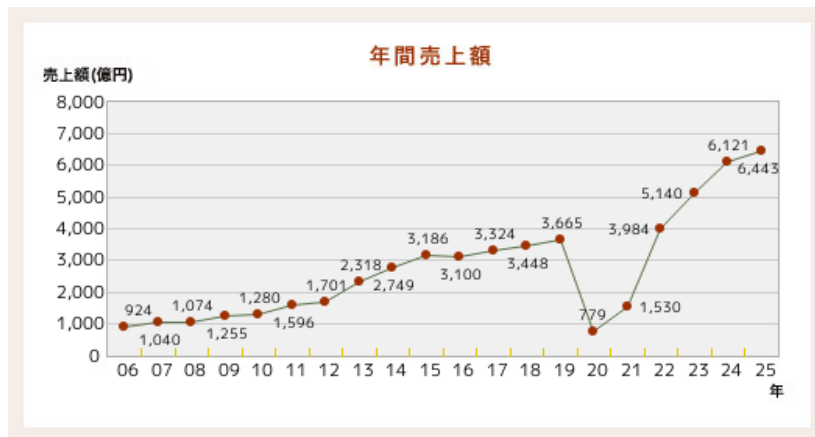


図1 国内ライブ・コンサート市場における年間売上額の推移（一般社団法人コンサートプロモーターズ協会「基礎調査推移表」より）

と共有しているという感覚そのものが、没入感を左右する独立した要因であり、これは録画・配信技術の高度化だけでは補えないことを意味する。したがって、音そのものの再現性だけでなく、ライブ体験に固有の空間的フィードバックを組み込むことで没入感の高い演奏体験の実現に有効であると考えられる。

また、Natalia Cooper ら（2018）は、仮想現実（VR）環境における体験の再現度を高めるため、視覚情報のみでは表現が難しい感覚を、音や触覚など別の感覚を用いた代替フィードバックとして提示する実験を行った。その結果、視覚刺激のみを提示した場合と比較して、音や触覚といった複数の異なる刺激を組み合わせると提示した場合の方が、利用者の没入感・関与感が有意に高まることを明らかにした [6]。つまり、複数の感覚情報を同時に提示する多感覚的フィードバックは、ユーザの体験における没入感の向上に有効であると考えられる。また、Human-Computer Interaction（HCI）分野の知見も同様の傾向を支持している。Martin ら（2024）は、ライブコンサート環境を対象とした研究において、観客が演奏を受動的に聴取するだけでなく、能動的に体験へ参加することが、会場全体の活気や観客の感情の高まりに寄与すると結論づけた [7]。よって、ユーザ自身が能動的に関与できる仕組みもまた、システムの没入感を高める要因として重要であると考えられる。

一方、音楽を自動で演奏する自動演奏技術の分野では、デジタル技術の発展により高精度な同期や楽曲再現が可能になっている。たとえば、Yamaha が提供する Disklavier ENSPIRE シリーズは、光学センサとサーボモータを用いて演奏情報を高精度に再現するとともに、鍵盤が実際に動作する様子をユーザへ視覚的にフィードバックを行う機能を備えている [8]。しかし、この視覚的フィードバックは、ピアノの鍵盤という限られた範囲でのみ提示されるものであり、その場に居合わせたユーザ1人のみが確認できる情報にとどまる。したがって、複数人が同じ空間にいる状況を想定すると、自動演奏技術によって提示される視覚的フィードバックは、空間全体には行き渡らないという限界を抱えていると言える。

以上を踏まえると、現状の自動演奏技術は、演奏そのものの精度は高い一方で、前述した「多感覚的フィードバック」と「空間全体への共有」という、没入感を高めるうえで重要な要素を十分に

は満たせていないという課題が生じる。特に、Kreuzer らの知見が示すとおり、同じ場を共有しているという空間的なフィードバックは、音質や演奏精度をいくら高めても代替できない、ライブ体験に固有の要素である。したがって、デジタル配信や高精度な自動演奏技術がどれほど発展しても、この空間的フィードバックを欠いたままでは、ライブ会場に匹敵する高い没入感を得ることはできないと結論づけられる。そこで本稿では、自動演奏技術に、ライブ体験に共通するこの空間的フィードバックを付加した楽器間通信システム「回転機構のレーザー受光と楽器間無線通信による同期・輪唱システム」を提案する。本システムは、ユーザの動作を検知する指揮デバイス、指揮デバイスからの情報に応じ回転する観覧車型の中継機デバイス、そして中継機からのレーザー光を受光し演奏を行う楽器デバイスによって構成される。指揮デバイスによるユーザの能動的な動作、観覧車型デバイスによる空間全体への視覚的なフィードバックの共有、そして高精度な音色の演奏による聴覚的な刺激を組み合わせることで、従来の自動演奏技術では実現が難しかった、会場全体を巻き込む没入感の高い演奏体験の提供を目指す。

1.2 類似技術・先行研究

本節では、本システムの独自性を明確にするため、音楽に視覚的なフィードバックを付加する既存技術を整理し、それぞれの特徴と限界を検討する。

第一に、DMX512 信号（以下、DMX 信号と表記する）を用いた照明制御システムが挙げられる。DMX 信号とは、舞台照明の操作卓と調光機との間でデータをやり取りするための通信規格であり、現在では舞台演出やコンサートにおいて、音楽に合わせた光量制御を行う目的で広く利用されている [9]。このシステムは、音楽という聴覚的な刺激に加えて光という視覚的な刺激を提示することで、観客の没入感を高めることができる。しかし、DMX 信号による照明制御は、あらかじめプログラムされた演出を再生する仕組みであるため、観客はその光の変化を一方的に受け取る、すなわち受動的に認識する立場にとどまる。前節で述べたとおり、能動的な関与が没入感を高める要因の一つであることを踏まえると、観客が演奏そのものに関与できない DMX 信号による演出には限界があると言える。この課題に対し、本提案では指揮デバイスを導入し、利用者が指揮という身体動作を通じて演奏そのものを能動的に制御できるようにすることで、身体的な関与に基づく没入感の向上を狙う。

第二に、物理的な操作によって音楽を制御するタンジブル・インターフェースの例として、ReacTable が挙げられる。ReacTable は、卓型のディスプレイ上に複数の物理モジュールを配置し、その配置や動きに応じて音を合成するシンセサイザの一種であり、利用者は身体的な物理操作を通じて演奏を制御するとともに、ディスプレイに表示される視覚的な情報によって演奏状態のフィードバックを受け取ることができる [10]。しかし、この視覚的フィードバックはディスプレイという限られた表示領域内で完結しており、操作を行う利用者本人以外の第三者や、会場全体へフィードバックを共有する仕組みとしては設計されていない。そこで本提案では、指揮デバイスに加えて観覧車型の中継機デバイスを導入する。観覧車型デバイスが演奏情報に応じて回転する様子を、利用者本人だけでなく周囲にいる第三者も視認できるようにすることで、演奏状態の把握や共有を空間全体に広げ、多感覚的なフィードバックを実現することを狙う。

以上のように、DMX 信号による照明制御は観客の能動的な関与を欠き、ReacTable のような既存のタンジブル・インターフェースは視覚的フィードバックが個人の操作範囲内にとどまるという、それぞれ異なる限界を抱えている。本提案は、指揮デバイスによる能動的な演奏制御という既存研究の知見と、観覧車型デバイスによる空間共有性という要素を組み合わせることで、これら二つの課題を同時に解決し、空間全体への多感覚的なフィードバックを通じて、利用者の没入感および関与感を高めることを目指す。

2 作成したシステムの概要

本節では、作成したシステムの概要について説明する。本システムは、ユーザの能動的な動作と連動した観覧車型中継機デバイスの回転およびレーザーの発光を通じて、演奏空間全体に多感覚なフィードバックを与えることで、従来の自動演奏技術における演奏インターフェースでは得られない高い没入感をユーザに提供することを目的として設計されている。本システムは、指揮デバイス、観覧車型の中継機デバイス、楽器デバイスという3つのデバイスから構成される。演奏開始後、ユーザがスライド式可変抵抗器を操作したうえで指揮デバイスを振ると、加速度センサが振り動作を検知し、中継機デバイスが

ここからは各デバイスの概要について説明する。

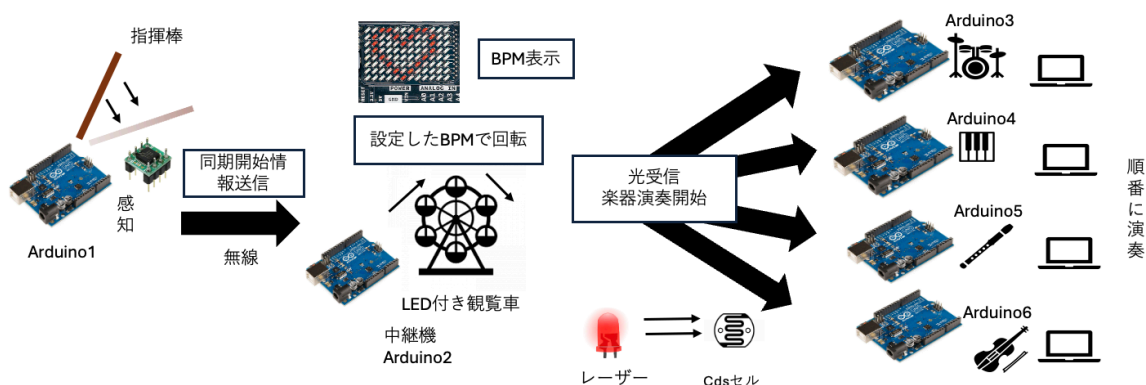


図2 全体の構成図

2.1 指揮デバイス

2.1.1 概要

まず、指揮デバイスについて説明する。指揮デバイスのシステム構成図を図3に示す。本デバイスは、演奏全体を統括するデバイスであり、主に以下の3つの機能を有する。

第一の機能は、演奏開始トリガーの送信である。本機能の設計について図4に示す。ユーザが指揮デバイスに搭載されたタクトスイッチを押下すると、中継機デバイスと楽器デバイスへ演奏開始トリガーを送信する。さらに、中継機デバイスの回転が開始される。

第二の機能は、楽曲のBPM変更である。本機能の設計について図5に示す。ユーザがスライド式可変抵抗器を操作した後、デバイスを振ると新しい設定BPM情報が中継機デバイスへ送信される。これにより、中継機デバイスのモータ回転速度がこの値に追従して変化する。そして、その回転に伴うレーザーの通過光を介して各楽器デバイスに情報が伝達され、BPMの変更される。また、楽曲のBPMは5段階に分けて変更できる。

第三の機能は、楽曲の音量変更である。本機能の設計について図6に示す。ユーザがスライド式可変抵抗器を操作した後、デバイスを振ると音量情報が各楽器デバイスへマルチキャスト送信されることで、音量が変更される。また、BPMと同様に音量も5段階に分けて変更できる。ここからは、外部設計と内部設計について説明をする。

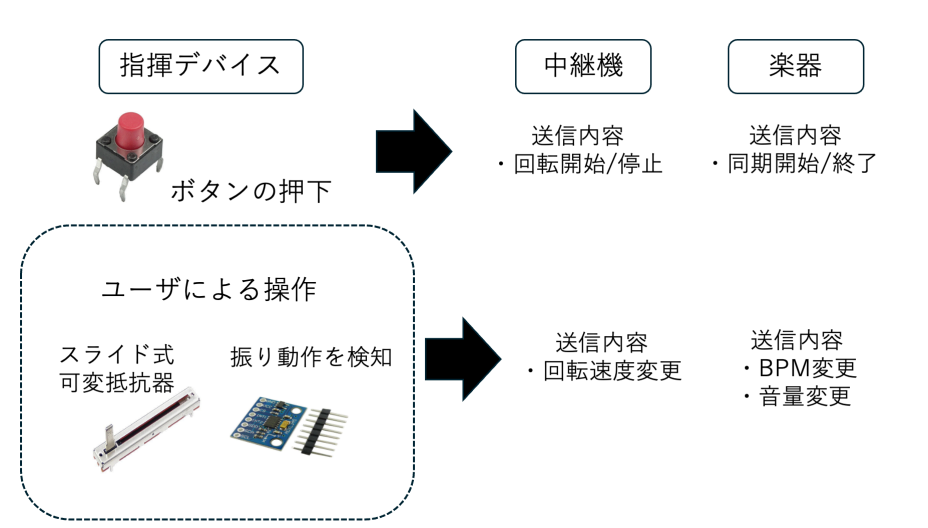


図3 指揮デバイスのシステム構成図

2.1.2 外部設計

本節では指揮デバイスの外部設計について説明する。まず、本デバイスを構成する部品について表1に示す。デジタル3軸加速度センサは指揮デバイスの動きの検知に使用し、スライド式可変抵抗器はBPMおよび音量変更に使用される。当初はデバイスの電力供給源として9Vのアルカリ電池を外部電源として使用することを検討していたが、電池なしでも正常動作が確認できたため、軽量化のためPCとのUSB接続から電力を供給するように変更した。Arduino内蔵の電圧レギュレータにより5Vおよび3.3Vを生成し、各モジュールへ安定した電力を供給する。また、指揮デバイスの回路図を図7に示す。加速度センサにはGY-291(ADXL345)を用いており、I2C通信によりArduinoと接続する。CSピンをArduinoの3.3Vに接続することでI2Cモードに設定し、さらにSDOピンをGNDに接続することでI2Cアドレスを0x53に固定している。また、SDAピンおよび

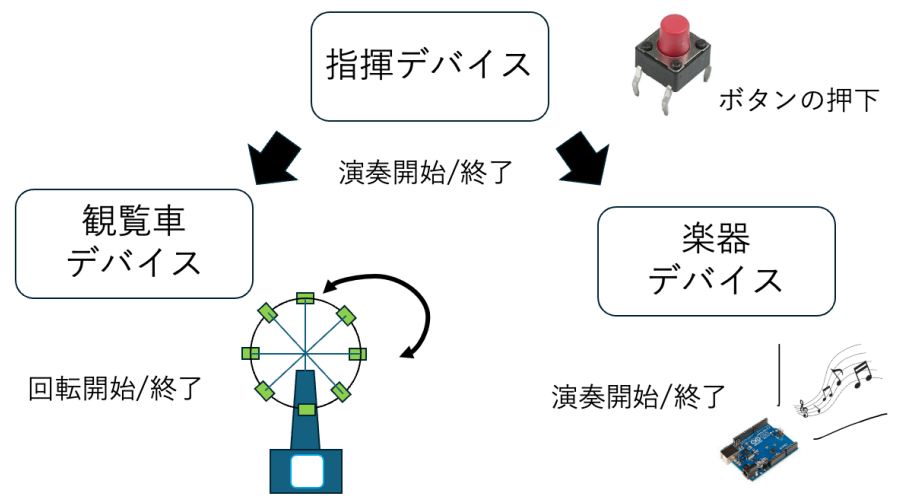


図 4 演奏開始トリガー送信機能設計

SCL ピンはそれぞれデータ通信線およびクロック信号線として Arduino の A4, A5 ピンに接続している。加えて、加速度が内部設定した閾値を超過した場合には、INT1 ピンから Arduino の D2 ピンへ割り込み信号が出力される。これにより、指揮デバイスの振り動作を検知し、モータ制御用 Arduino との UDP 通信を開始する。

さらに、演奏開始および終了信号の送信に用いるボタン入力判定回路を実装する。ボタンの誤判定を防止するため 10k Ω のプルアップ抵抗を用いた回路を構成する。ボタンが押されていない状態では、入力ピン D3 はプルアップ抵抗を介して 5V に接続されるため HIGH 状態となる。一方、ボタン押下時には回路が GND に短絡され、入力電位が LOW に遷移する。指揮デバイスのプログラムでは D3 ピンの状態変化を常時監視することで、演奏開始および終了操作を判定する。

表 1 指揮デバイスの構成部品

Arduino UNO R4 WiFi	-	1
スライド式可変抵抗器 10 k Ω	-	2
デジタル 3 軸加速度センサ	GY-291	1
タクトスイッチ	DTS-63	1
抵抗器 10k Ω	-	1
ジャンパーワイヤー	-	適宜

図～～～に実際に作成した指揮デバイスを示す。実際の指揮棒のようにユーザが片手で振って操作できるよう、細長い筒状の外装を採用した。また、内部にはブレッドボードに加速度センサ、スライド式可変抵抗器、タクトスイッチを設置する。デバイスのユーザビリティを向上させるため、ユーザが操作するスライダー部分やボタンは外部へ露出させる構造を採用し、さらにデバイスの先端部に加速度センサを設置することで操作時の重心を安定させる。

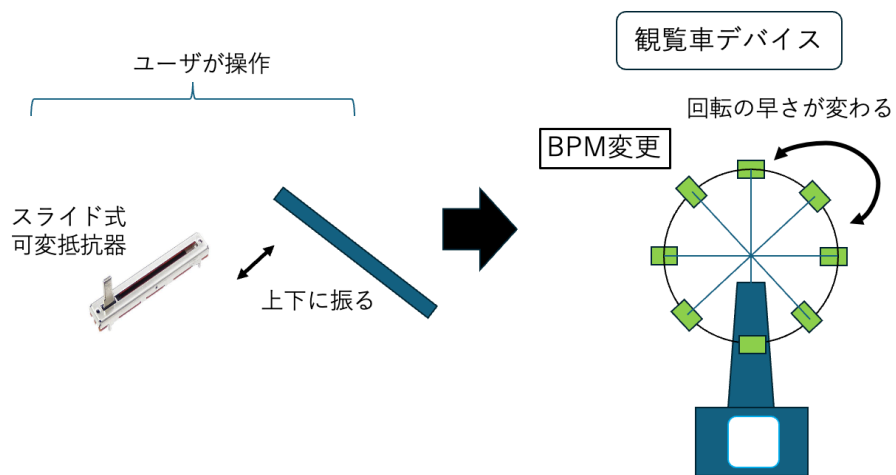


図 5 BPM 変更機能設計

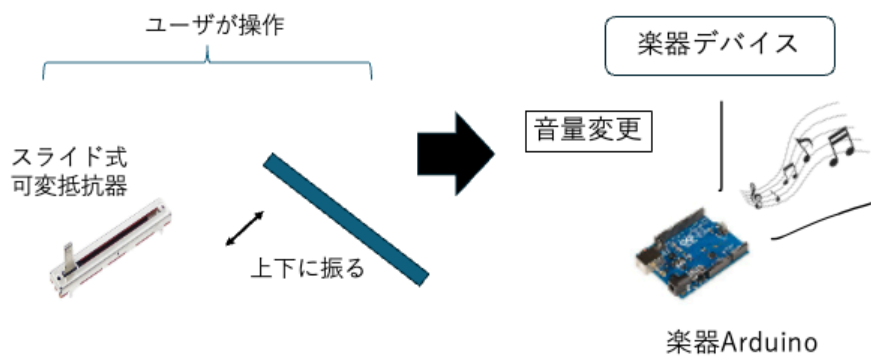


図 6 音量変更機能設計

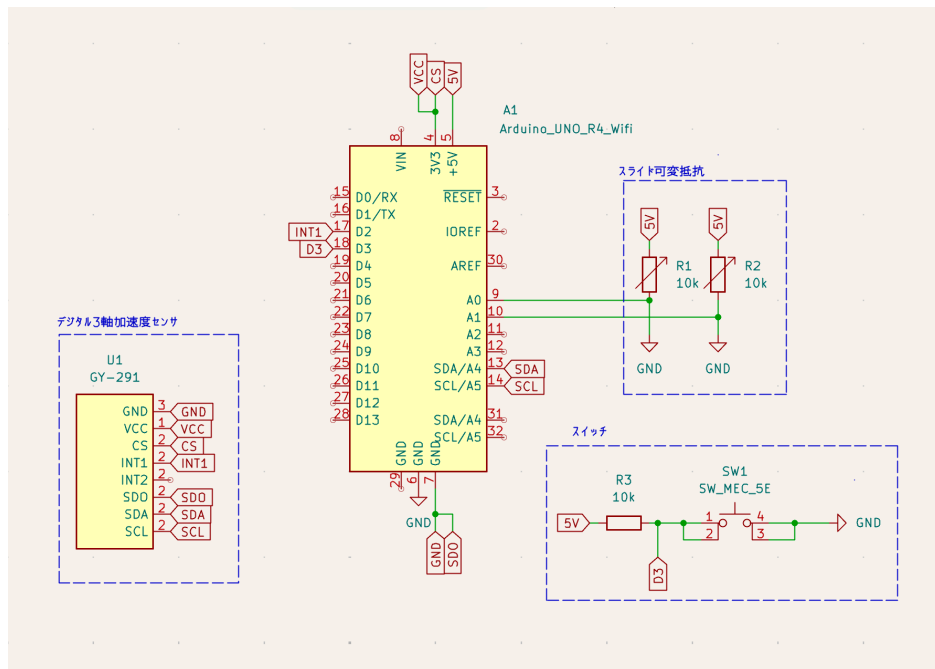


図7 指揮デバイスの回路図

2.1.3 内部設計

本節では、指揮デバイスの内部設計について説明する。まず、指揮デバイス全体の内部動作を明確にするため、シーケンス図を図8に示す。ユーザが指揮デバイスを上下に振ると加速度センサが動作を検知しArduinoに送信される。この時、値が閾値を超過した場合、観覧車デバイスと無線通信を開始し観覧車デバイスが送信した演奏情報を基に回転する。また、ボタンを押下した時、各楽器デバイスへ楽曲の初期BPMおよび音量情報が送信されることで演奏が開始される。

次に、指揮デバイスの制御プログラムのファイル構成を示す。表2の通り、各ファイルは機能ごとにクラス化されている。

表2 指揮デバイスのファイル構成

ファイル名	概要
Controller_simultaneous.ino	setup/loopによる全体制御、ボタン入力の判定
AccManager.h / AccManager.cpp	加速度センサ (ADXL345) の初期化、振り動作の検知、I2Cバス復旧処理
PotManager.h / PotManager.cpp	スライド式可変抵抗器2系統の移動平均処理およびBPM・音量段階の判定
SyncManager.h / SyncManager.cpp	Wi-Fi/UDP通信の初期化、各デバイスへの冗長送信処理

Controller_simultaneous.inoのloop関数は、ボタン判定を行うbutton関数、可変抵抗器を処理するPotManager、加速度センサを処理するAccManagerを毎ループ順に呼び出す構成となっている。

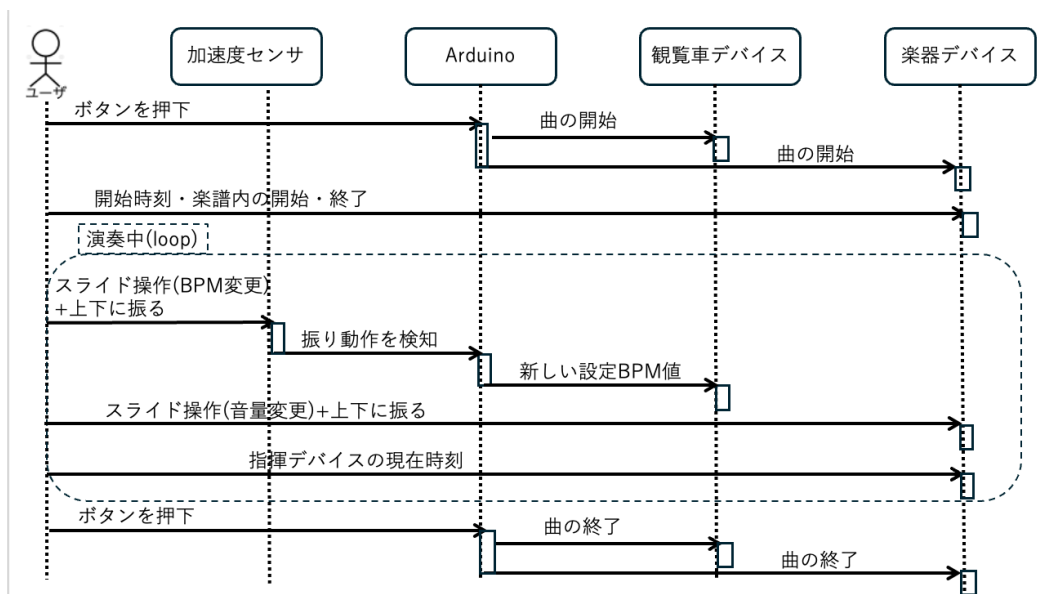


図 8 指揮デバイスのシーケンス図

loop 関数をコード 1 に示す。プログラム全文は付録に示す。

コード 1 Controller_simultaneous.ino の loop 関数

```

1 void loop() {
2   button();
3   pot.update();
4   acc.update(sync, pot);
5 }
  
```

ここからは、ボタン入力の判定、加速度センサの制御、可変抵抗器を用いた BPM・音量読み取り機能の順に、使用する変数・関数の仕様と処理内容について説明する。

まず、指揮デバイスにおけるボタン入力の判定処理について説明する。ボタンの誤判定を防ぐため、プルアップ抵抗を用いた回路を構成しており、ボタンが押されていない状態ではデジタルピンに 5V が印加され、押下時には回路が GND に短絡されることでピンの電位が LOW へ遷移する。指揮デバイスのプログラムでは、このデジタルピンの電圧状態の変化を常時監視することでボタンの押下を判定する。さらに、チャタリングによる誤判定や冗長パケットによる誤作動を防止するため、押下検知後に一定時間の待機処理を設けるとともに、演奏状態管理フラグ (isButtonPlaying) を用いたトグル制御を導入している。これにより、同一の物理ボタンのみで演奏の開始と終了を交互に切り替えることが可能となる。ボタン入力の判定で使用する変数の仕様を表 3 に示す。

表 3 ボタン入力の判定で使用する変数の仕様一覧

変数名	型	説明
input_PIN	const uint8_t	ボタンの状態を読み取るデジタルピン番号を定義
switch_status	bool	現在読み取ったボタンピンの状態を格納
before_switch_status	bool	前回読み取ったボタンピンの状態を格納（立下り検知に使用）
isButtonPlaying	bool	演奏状態管理フラグ。トグル制御により演奏中(true)／停止中(false)を保持

続いて、ボタン入力の判定で使用する関数の仕様を表 4 に示す。

表 4 ボタン入力の判定で使用する関数の仕様一覧

関数名	説明
button()	input_PIN の立下り（switch_status が 0 かつ before_switch_status が 1）を検知すると isButtonPlaying をトグルし、開始時は現在の BPM 段階番号を付与して SyncManager の sendStart 関数を、終了時は sendEnd 関数を呼び出すことで、各楽器機・観覧車デバイスへ演奏開始・終了信号を送信する。送信後は 300 ms の待機によりチャタリングを防止する。

button 関数はこの 1 関数のみで構成されるため、以下では処理の流れに沿って実際のコードを 2 箇所に分けて説明する。まず、立下りエッジの検知部分について説明する。switch_status に現在のピン状態を読み込み、前回値 before_switch_status と比較することで、HIGH → LOW へ変化した瞬間（ボタンが押された瞬間）のみを検知する。該当部分をコード 2 に示す。

コード 2 button 関数（立下りエッジの検知部分）

```

1  switch_status = digitalRead(input_PIN);
2  if (switch_status == 0 && before_switch_status == 1) {
3      ...
4  }
5  before_switch_status = switch_status;

```

次に、立下りを検知した際の状態トグルと送信処理について説明する。isButtonPlaying が true（演奏中）であれば sync.sendEnd() を呼び出して演奏を終了し、false（停止中）であれば現在の BPM 段階（pot.getBpmStep()）を引数として sync.sendStart() を呼び出して演奏を開始する。いずれの場合も isButtonPlaying の値を反転させたくえて、delay(300) により 300 ms 待機することでチャタリングを防止する。該当部分をコード 3 に示す。

コード 3 button 関数 (状態トグルと送信処理部分)

```
1  if (isButtonPlaying) {
2      sync.sendEnd();
3      isButtonPlaying = false;
4  } else {
5      uint8_t step = pot.getBpmStep();
6      sync.sendStart(step);
7      isButtonPlaying = true;
8  }
9  delay(300); // チャタリング防止
```

なお、sendStart・sendEndの内部では、SyncManagerクラスが複数台のデバイスへラウンドロビン方式による冗長送信を行っている。この通信処理の詳細な実装は付録に示すプログラム全文を参照されたい。

次に、指揮デバイスにおける加速度センサの制御処理について説明する。指揮デバイスは、加速度センサを用いてユーザが指揮棒を振ったことを検知し、演奏情報の更新を行う。センサ値の急激な変化を読み取ることで動作を判定し、誤検知防止機能を設けることで安定した動作を実現している。処理の流れは、差分算出と判定、多重検知の防止、状態の反転、事後処理の4段階に大別される。まず差分算出と判定では、現在の合成加速度 currentAccVal と前回値 lastAccVal との差の絶対値を求める。この差分が検知閾値 accThreshold を超過し、かつ前回検知から shakeInterval 以上の時間が経過している場合に、指揮棒を振ったと判定する。多重検知の防止では、一度振る動作を検知すると lastDetectTime を現在時刻に更新し、以降 shakeInterval が経過するまでは加速度の変化を無視することで、一連の動作による複数回の誤検知を防止する。状態の反転では、振る動作を検知するたびに isStarted の bool 値を反転させる。事後処理として、次回の判定に備えて currentAccVal の値を lastAccVal へ代入し、前回のサンプリング時刻を更新する。また、振る動作の確定時には、可変抵抗器側で設定値の変更を検知したフラグ bpmFrag または volFrag (2.1.3 項で説明する) が有効になっている場合、SyncManager クラスを介してその情報の送信指示を行う。さらに、I2C 通信の異常を検知した場合には自動復旧を行う機能を備えている。加速度センサの制御で使用する変数の仕様を表 5 に示す。

表 5 加速度センサの制御で使用する変数の仕様一覧

変数名	型	説明
currentAccVal	float	加速度センサから算出した合成加速度の値を格納
lastAccVal	float	変化量算出のための直前の加速度センサ値を格納
accThreshold	const float	振ったと判断するための変化量の閾値を定義
lastDetectTime	unsigned long	前回振ったことを検知した時間を ms 単位で格納
shakeInterval	const unsigned long	連続検知を防止するための待機時間を定義
lastSampleTime	unsigned long	前回センサ値をサンプリングした時間を ms 単位で格納
ACC_SAMPLE_INTERVAL	const unsigned long	加速度センサ値をサンプリングする周期を定義
isStarted	bool	楽器の演奏状態を保持 (true: 演奏中/false: 停止中)
ADXL345_ADDR	const uint8_t	加速度センサ (ADXL345) の I2C アドレスを定義
REG_POWER_CTL	const uint8_t	加速度センサの電源管理レジスタのアドレスを定義
REG_DATA_FORMAT	const uint8_t	測定範囲設定用レジスタのアドレスを定義
REG_DATA0	const uint8_t	測定値が格納される先頭レジスタのアドレスを定義
lastRecoverMs	unsigned long	I2C 復旧処理の連続実行を防ぐための最終復旧時刻を保持

続いて、加速度センサ制御 (AccManager クラス) で使用する関数の仕様を表 6 に示す。

表 6 加速度センサ制御 (AccManager クラス) で使用する関数の仕様一覧

関数名	説明
setupADXL345()	ADXL345 の電源投入と測定レンジの設定を行い、初期の加速度値を lastAccVal に格納する初期化関数
readAccMagnitude()	ADXL345 から X/Y/Z 軸の生の値を取得し、2 乗和の平方根を返す。I2C 通信異常を検知した場合は直前の加速度値を返す
recoverI2C()	I2C バスのロックアップ異常を検出した場合に、SCL 線のクロッキングによるバスクリアと STOP 条件の生成、Wire およびセンサの再初期化によって通信を復旧する
update(sync, pot)	ACC_SAMPLE_INTERVAL ごとに加速度値を取得し、差分算出・多重検知防止判定・状態の反転・事後処理までの一連の振る動作検知処理を行う。loop 内で毎回呼び出す
updateShakedInfo(sync, pot)	振る動作の確定時に呼び出され、PotManager の bpmFrag / volFrag が立っている場合に SyncManager 経由で BPM・音量の段階番号を送信する

以下では、表 6 に示した順に、各関数の処理内容を実際のコードとともに説明する。はじめに、setupADXL345 関数について説明する。この関数は、Wire.begin による I2C 通信の開始後、ADXL345 の POWER_CTL レジスタに Measure ビットを書き込むことでセンサを測定モードへ移行させ、DATA_FORMAT レジスタを初期値のまま設定する。setupADXL345 関数をコード 4 に示す。

コード 4 AccManager::setupADXL345 関数

```

1 void AccManager::setupADXL345() {
2     Wire.begin();
3
4     Wire.beginTransmission(ADXL345_ADDR);
5     Wire.write(REG_POWER_CTL);
6     Wire.write(0x08); // Measure bit ON
7     Wire.endTransmission();
8
9     Wire.beginTransmission(ADXL345_ADDR);
10    Wire.write(REG_DATA_FORMAT);
11    Wire.write(0x00);
12    Wire.endTransmission();
13
14    lastAccVal = readAccMagnitude();
15 }
```

次に、加速度の実測値を取得する readAccMagnitude 関数について説明する。この関数は、DATA0 レジスタから連続する 6 バイト (X, Y, Z 軸それぞれ 2 バイトの符号付き整数) を requestFrom で読み出し、3 軸のベクトル合成値 (合成加速度) を平方根の計算により求めて返り値とする。readAccMagnitude 関数をコード 5 に示す。

コード 5 AccManager::readAccMagnitude 関数

```
1 float AccManager::readAccMagnitude() {
2   Wire.beginTransmission(ADXL345_ADDR);
3   Wire.write(REG_DATA0);
4   if (Wire.endTransmission(false) != 0) {
5     recoverI2C();
6     return lastAccVal;
7   }
8
9   uint8_t n = Wire.requestFrom(ADXL345_ADDR, (uint8_t)6, (uint8_t)true);
10  if (n < 6) {
11    recoverI2C();
12    return lastAccVal;
13  }
14
15  int16_t rawX = (int16_t)((Wire.read() | (Wire.read() << 8));
16  int16_t rawY = (int16_t)((Wire.read() | (Wire.read() << 8));
17  int16_t rawZ = (int16_t)((Wire.read() | (Wire.read() << 8));
18
19  return sqrtf((float)rawX*rawX + (float)rawY*rawY + (float)rawZ*rawZ);
20 }
```

ここで、Wire.endTransmission(false)がエラーを返した場合、またはrequestFromで6バイト読み出せなかった場合には、I2Cバスがロックアップ（スレーブがSDA線を握ったまま停止する現象）していると判断し、recoverI2C関数を呼び出したうえで前回値lastAccValを返すことで、誤ったシェイク検知（ニセshake）を防いでいる。続いて、recoverI2C関数について説明する。この関数は、SCL線を9回手動でトグルさせてSDA線の握りを解放したうえでSTOP条件を生成し、Wireを再初期化してADXL345を再設定することでバスを復旧させる。lastRecoverMsは、この復旧処理およびログ出力が短時間に連続して実行されることを防ぐスロットル用の時刻管理に用いられる。recoverI2C関数をコード6に示す。

コード 6 AccManager::recoverI2C 関数

```
1 void AccManager::recoverI2C() {
2   unsigned long now = millis();
3   if (now - lastRecoverMs >= 1000) { // ログのスロットル
4     lastRecoverMs = now;
5   }
6
7   Wire.end();
8
9   // バスクリア: SCLを9クロック叩いて握られたSDAを解放
10  pinMode(SDA, INPUT_PULLUP);
11  pinMode(SCL, OUTPUT);
12  for (int i = 0; i < 9; i++) {
13    digitalWrite(SCL, LOW); delayMicroseconds(5);
14    digitalWrite(SCL, HIGH); delayMicroseconds(5);
```

```

15 }
16 // STOP条件 (SDA: LOW→HIGH while SCL HIGH)
17 pinMode(SDA, OUTPUT);
18 digitalWrite(SDA, LOW); delayMicroseconds(5);
19 digitalWrite(SCL, HIGH); delayMicroseconds(5);
20 digitalWrite(SDA, HIGH); delayMicroseconds(5);
21
22 // Wire再初期化&センサ再設定
23 Wire.begin();
24 Wire.beginTransmission(ADXL345_ADDR);
25 Wire.write(REG_POWER_CTL); Wire.write(0x08);
26 Wire.endTransmission();
27 Wire.beginTransmission(ADXL345_ADDR);
28 Wire.write(REG_DATA_FORMAT); Wire.write(0x00);
29 Wire.endTransmission();
30 }

```

次に、update 関数について説明する。この関数は loop 内で毎回呼び出され、ACC_SAMPLE_INTERVAL (20 ms) ごとに加速度の実測値を取得し、前回値との差 diff を求める。diff が検知閾値 accThreshold を超過し、かつ前回検知から shakeInterval (750 ms, 多重検知防止用) が経過している場合にのみ、振り動作として検知し updateShakedInfo 関数を呼び出す。update 関数をコード 7 に示す。

コード 7 AccManager::update 関数

```

1 void AccManager::update(SyncManager& sync, PotManager& pot) {
2     unsigned long now = millis();
3     if (now - lastSampleTime >= ACC_SAMPLE_INTERVAL) {
4         lastSampleTime = now;
5         currentAccVal = readAccMagnitude();
6         float diff = fabsf(currentAccVal - lastAccVal);
7
8         bool overThreshold = (diff > accThreshold);
9         bool intervalPassed = ((now - lastDetectTime) >= shakeInterval);
10
11         if (overThreshold && intervalPassed) {
12             lastDetectTime = now;
13             isStarted = !isStarted;
14             updateShakedInfo(sync, pot);
15         }
16         lastAccVal = currentAccVal;
17     }
18 }

```

最後に、updateShakedInfo 関数について説明する。この関数は、振り動作の検知をトリガとして、PotManager が保持する bpmFrag・volFrag フラグ (可変抵抗器の値が変化した場合に立つフラグ、2.1.3 項で説明する) を確認し、フラグが立っている項目についてのみ SyncManager 経由で BPM または音量の変更情報を送信する。すなわち、実際に無線送信が発生するのは、可変抵抗器

を操作した後にデバイスを振った場合に限られる。updateShakedInfo 関数をコード 8 に示す。

コード 8 AccManager::updateShakedInfo 関数

```
1 void AccManager::updateShakedInfo(SyncManager& sync, PotManager& pot) {
2     if (pot.bpmFrag) {
3         sync.sendBPM(pot.getBpmStep());
4         pot.bpmFrag = false;
5     }
6     if (pot.volFrag) {
7         sync.sendVolume(pot.getVolStep());
8         pot.volFrag = false;
9     }
10 }
```

最後に、指揮デバイスにおける可変抵抗器を用いた BPM・音量読み取り機能について説明する。指揮デバイスでは、演奏の要素である BPM と音量を、ユーザが操作できるように可変抵抗器を用いて 5 段階で調整する。アナログ入力値の特性を考慮したデータ処理を行うことで、安定した制御環境を構築している。アルゴリズムは、サンプリングと移動平均化、範囲分割、パラメータの抽出と出力の 3 段階に分けられる。サンプリングと移動平均化では、SAMPLE_INTERVAL ごとに BPM および音量変更用の可変抵抗器の電圧値を、配列 bpmSamples・volSamples に sampleSize 分だけ読み取り保存する。このとき、配列の各要素は書き込み位置 (bpmReadIndex・volReadIndex) が指す最も古い値から順に上書きされ、上書きの直前に該当要素の値を合計 bpmTotal・volTotal からあらかじめ減算し、新たに読み取った値を加算することで、配列全体を読み直すことなく常に最新の合計値を保持する（移動合計）。配列が一巡し全要素が新しい値に更新された時点 (bpmBufferFull・volBufferFull が true になった時点) 以降、この合計値を sampleSize で除算した移動平均値を averageBpmVal・averageVolVal に格納する。範囲分割では、移動平均化された値を STEP_BOUNDARIES[] で定義した 4 つの閾値と比較し、1～5 の段階番号を currentBpmStep・currentVolStep に格納する。パラメータの抽出と出力では、算出した段階が直前の段階 (prevBpmStep・prevVolStep) から変化した場合にのみ、公開フラグ bpmFrag・volFrag を立てる。外部からは getBpmStep()・getVolStep() を介して現在の段階を取得し、前述の加速度センサ制御における棒振り動作の確定時にこれらのフラグを参照して各楽器機への送信を行う。なお、音量用可変抵抗器は配線の都合上、回した方向とセンサ値の増減が逆になるため、getVolStep() は 6-currentVolStep を返すことで表示上の段階を反転させる。BPM・音量読み取り機能で使用する変数の仕様を表 7 に示す。

表 7: BPM・音量読み取り機能で使用する変数の仕様一覧

変数名	型	説明
PIN_BPM	const int	BPM 変更用の可変抵抗器を接続するアナログ入力ピン番号を定義

次ページに続く

表 7 続き

変数名	型	説明
PIN_VOL	const int	音量変更用の可変抵抗器を接続するアナログ入力ピン番号を定義
SAMPLE_INTERVAL	const unsigned long	可変抵抗器の値をサンプリングする周期を定義
sampleSize	const int	移動平均算出のためのサンプル数を定義
STEP_BOUNDARIES[]	const int	段階判定に用いる 4 つの閾値を保持する配列
lastSampleTime	unsigned long	前回サンプリングを行った時間を格納
bpmSamples[]	int	BPM 用のサンプル値を保持するための配列
volSamples[]	int	音量用のサンプル値を保持するための配列
bpmReadIndex	int	bpmSamples[] 内での現在の書き込み位置を管理するインデックス
volReadIndex	int	volSamples[] 内での現在の書き込み位置を管理するインデックス
bpmBufferFull	bool	bpmSamples[] が一巡し、移動平均の算出が有効になったかを保持
volBufferFull	bool	volSamples[] が一巡し、移動平均の算出が有効になったかを保持
bpmTotal	long	bpmSamples[] 内の累積合計を格納
volTotal	long	volSamples[] 内の累積合計を格納
averageBpmVal	float	算出された BPM 用のアナログ入力の移動平均値を格納
averageVolVal	float	算出された音量用のアナログ入力の移動平均値を格納
currentBpmStep	uint8_t	算出された BPM の 5 段階のレベル (1～5) を格納
currentVolStep	uint8_t	算出された音量の 5 段階のレベル (1～5) を格納
prevBpmStep	int	段階の変化検知のための直前の BPM 段階を格納

次ページに続く

表 7 続き

変数名	型	説明
prevVolStep	int	段階の変化検知のための直前の音量段階を格納
bpmFrag	bool	BPM 段階が変化したことを示す公開フラグ。シェイク確定時の連動送信に使用
volFrag	bool	音量段階が変化したことを示す公開フラグ。シェイク確定時の連動送信に使用

続いて、可変抵抗器読み取り機能（PotManager クラス）で使用する関数の仕様を表 8 に示す。

表 8 可変抵抗器読み取り機能（PotManager クラス）で使用する関数の仕様一覧

関数名	説明
update()	SAMPLE_INTERVAL ごとに BPM・音量それぞれの可変抵抗器の値をサンプリングし、移動合計・移動平均・段階算出を行う。loop 内で毎回呼び出す
calcStep(val)	0～1023 の移動平均値を STEP_BOUNDARIES[] と比較し、1～5 のいずれかの段階を返す
bpmUpdatePotentiometers()	BPM 用可変抵抗器の値を読み取り移動合計・移動平均を更新し、段階が変化した場合に true を返す
volUpdatePotentiometers()	音量用可変抵抗器の値を読み取り移動合計・移動平均を更新し、段階が変化した場合に true を返す
getBpmStep()	現在の BPM 段階（currentBpmStep）を外部に返すゲッター
getVolStep()	現在の音量段階を外部に返すゲッター。配線特性による値の反転を補正し、6-currentVolStep を返す

以下では、表 8 に示した順に、各関数の処理内容を実際のコードとともに説明する。はじめに、update 関数について説明する。この関数は loop 内で毎回呼び出され、SAMPLE_INTERVAL（20 μ s）ごとに bpmUpdatePotentiometers 関数と volUpdatePotentiometers 関数を呼び出す。update 関数をコード 9 に示す。

コード 9 PotManager::update 関数

```

1 void PotManager::update() {
2     unsigned long now = micros();
3     if (now - lastSampleTime >= SAMPLE_INTERVAL) {
4         lastSampleTime = now;

```

```

5     if (bpmUpdatePotentiometers()) bpmFrag = true;
6     if (volUpdatePotentiometers()) volFrag = true;
7 }
8 }

```

次に、bpmUpdatePotentiometers 関数について説明する。この関数は、リングバッファ（配列 bpmSamples, 添字 bpmReadIndex）を用いた移動平均処理により、analogRead で取得したアナログ値のノイズを平滑化する。具体的には、配列の最も古いサンプルを合計値 bpmTotal から減算し、新たに取得した値を加算したうえで書き込み位置を進める。sampleSize（10 サンプル）分のバッファが一巡した時点から、合計値をサンプル数で割った移動平均 averageBpmVal を算出し、calcStep 関数で段階へ変換したうえで、前回段階から変化していれば true を返す。bpmUpdatePotentiometers 関数をコード 10 に示す。

コード 10 PotManager::bpmUpdatePotentiometers 関数

```

1 bool PotManager::bpmUpdatePotentiometers() {
2     bpmTotal -= bpmSamples[bpmReadIndex];
3     bpmSamples[bpmReadIndex] = analogRead(PIN_BPM);
4     bpmTotal += bpmSamples[bpmReadIndex];
5     bpmReadIndex = (bpmReadIndex + 1) % sampleSize;
6     if (bpmReadIndex == 0) bpmBufferFull = true;
7     if (!bpmBufferFull) return false;
8
9     averageBpmVal = (float)bpmTotal / sampleSize;
10    currentBpmStep = calcStep(averageBpmVal);
11
12    if (currentBpmStep != prevBpmStep) {
13        prevBpmStep = currentBpmStep;
14        return true;
15    }
16    return false;
17 }

```

続いて、calcStep 関数について説明する。この関数は、bpmUpdatePotentiometers 関数および volUpdatePotentiometers 関数から呼び出され、得られた移動平均値を 1～5 の 5 段階に離散化する。STEP_BOUNDARIES 配列（300, 500, 650, 818）に対する大小比較によって段階を決定しており、境界値の間隔を均等にしていない理由は、可変抵抗器の実測特性に合わせて各段階の操作しやすさを調整しているためである。calcStep 関数をコード 11 に示す。

コード 11 PotManager::calcStep 関数

```

1 int PotManager::calcStep(float val) {
2     if (val <= STEP_BOUNDARIES[0]) return 1;
3     else if (val <= STEP_BOUNDARIES[1]) return 2;
4     else if (val <= STEP_BOUNDARIES[2]) return 3;
5     else if (val <= STEP_BOUNDARIES[3]) return 4;
6     else return 5;
7 }

```

次に、volUpdatePotentiometers 関数について説明する。この関数は bpmUpdatePotentiometers 関数と同様の移動平均処理を行うが、対象のピンおよび内部変数が音量用（PIN_VOL, volSamples 等）に置き換わっている点のみが異なり、処理の骨格は共通している。volUpdatePotentiometers 関数をコード 12 に示す。

コード 12 PotManager::volUpdatePotentiometers 関数

```
1 bool PotManager::volUpdatePotentiometers() {
2     volTotal -= volSamples[volReadIndex];
3     volSamples[volReadIndex] = analogRead(PIN_VOL);
4     volTotal += volSamples[volReadIndex];
5     volReadIndex = (volReadIndex + 1) % sampleSize;
6     if (volReadIndex == 0) volBufferFull = true;
7     if (!volBufferFull) return false;
8
9     averageVolVal = (float)volTotal / sampleSize;
10    currentVolStep = calcStep(averageVolVal);
11
12    if (currentVolStep != prevVolStep) {
13        prevVolStep = currentVolStep;
14        return true;
15    }
16    return false;
17 }
```

最後に、getBpmStep 関数および getVolStep 関数について説明する。いずれも PotManager.h 内でインライン定義されたゲッター関数であり、外部（AccManager クラス）から現在の段階番号を取得するために用いられる。getVolStep 関数では、可変抵抗器を回転させる向きと、音量として直感的に理解しやすい段階の増減方向を一致させるため、6 - currentVolStep という変換を行っている点が getBpmStep 関数と異なる。両関数をコード 13 に示す。

コード 13 PotManager::getBpmStep 関数・getVolStep 関数

```
1 uint8_t getBpmStep() const { return currentBpmStep; }
2 uint8_t getVolStep() const { return 6 - currentVolStep; }
```

2.2 中継機デバイス

2.2.1 概要

次に、中継機デバイスについて説明する。中継機デバイスの構成図を図 9 に示す。本デバイスは、指揮デバイスからの情報を受け取り中継機デバイスの回転を作動させ、現在の BPM を表示するデバイスであり、主に以下の 2 つの機能を有する。

第一の機能は、上記で述べた BPM に合わせて回転し楽曲の BPM を制御する機能である。デバイスに設置してあるレーザー受光のテンポで BPM を管理し、楽曲のテンポを視覚的にも理解できるようにする。

第二の機能は，Arduino Uno R4 WiFi に付属している LEDMatrix に BPM を表示する機能である．ここに指揮デバイスから受け取った BPM を表示し，視覚的に現在流れている楽曲の BPM を理解できるようにする．ここからは，外部設計と内部設計について説明する．

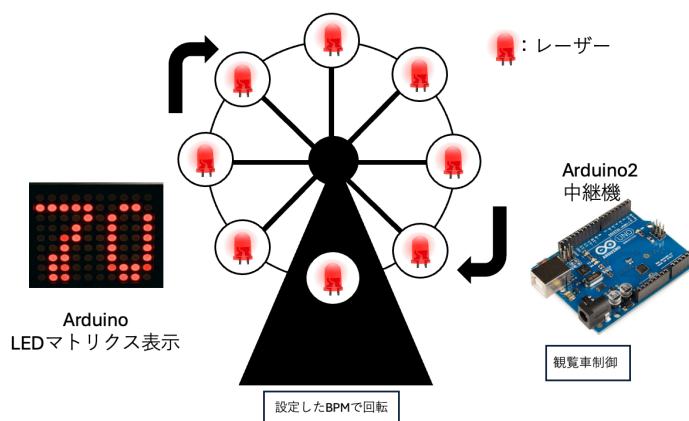


図 9 中継機デバイスの構成図

2.2.2 外部設計

本節では中継機デバイスの外部設計について説明する．まず，本デバイスを構成する部分について表 9 に示す．この内，個別で購入する必要があるものはハイパワーギヤーボックス，ブレッドボード用 DC ジャック DIP 化キット，ボタン電池，ボタン電池ホルダー，QI ケーブル，有極コンデンサーである．

また，中継機の回路図を図 10 に示す．モータードライバーに Arduino D5, D6 ピンを接続し，PWM 制御を行えるようにする．モータードライバーへの電力供給は当初，外部電源として 9V のアルカリ電池を検討していたが，Arduino の 5V からの電力供給と DC ジャックからの AC アダプター 5V1A による外部給電により，アルカリ電池が不要であることが実装時に確認されたため除外した．また，モータードライバー及び DC ジャックはピンヘッダーのはんだ付けを行う．加えて，ギアボックスは $0.01\mu\text{F}$ のコンデンサーと直接はんだ付けを行う．ギアボックスと並列に接続された $0.01\mu\text{F}$ のコンデンサは，モータのブラシ接点で発生する電気ノイズを吸収し，制御信号や周辺回路への影響を低減する役割を果たしている．

次に，観覧車の設計について説明する．作成するために，3D プリンタを用いて作成する．実際に設計に使用するモデルデータを図 11 から図 16 に示している．観覧車の直径は約 16cm とし，ボタン電池付きレーザーを 45 度間隔で配置する．そして，観覧車を支える左右の支柱および土台を設計する．観覧車本体は，以下のパーツに分割して設計を行う．

- ・土台 観覧車全体を支える部分であり，内部を空洞化させる．また，支柱を土台に差し込み，固定する役割も持つ．実際に設計する部品は図 11 の通りである．

表 9 中継機デバイスを構成する部品

機材	品番	個数 [個]
Arduino Uno R4 WiFi	-	1
ルーター	-	1
ブレッドボード	-	1
赤色ドットレーザーモジュール	FU650AD5-C6	8
ボタン電池基板取付用ホルダー (CR2477 用)	CH031-2477LF	4
ボタン電池	CR2477	4
スライドスイッチ	SS-12D00-G5	4
モータードライバーモジュール	AE-DRV8835-S	1
ハイパワーギヤーボックス HE	72003	1
ジャンパーワイヤー	-	適宜
ブレッドボード用 2.1mm 標準 DC ジャック DIP 化キット	AE-DC-POWER-JACK- DIP	1
超小型スイッチング AC アダプター 5 V 1 A	-	1
QI ケーブル 2P-1Px2	311-262	1
抵抗器 10 Ω	-	1
コンデンサー 0.01 μF	-	1
コンデンサー 0.047 μF	-	1
有極性コンデンサー 100 μF	-	1

- 支柱 観覧車の回転軸を左右から支える部品である。回転軸を安定して保持し、観覧車が滑らかに回転できるようにする。回転軸を上から挿入する形であり、軸が折れないようギアボックスモータに繋ぐ側の支柱の支える穴を大きくしている。実際に設計する部品は図 12 の通りである。
- 回転軸 ギアボックスモータの回転を観覧車本体へ伝達する部品である。モータと接続し、観覧車全体を回転させる役割を持つ。支柱に固定できるよう、支柱の位置の軸部をへこませて固定できるようにしている。実際に設計する部品は図 13 の通りである。
- 回転リング 観覧車の円形部分であり、レーザー側とその反対側で 2 個作成する。モータによって回転し、レーザーによる光信号送信を行う部分でもある。角度 45 度おきにレーザーを設置する穴を用意し、そこにレーザーをはめ込み固定する。もう一つの円盤に電池基板を設置し電力供給する。実際に設計する部品は図 14, 15 の通りである。
- 受光部 レーザーの延長線上に設置する照度センサーの受光部である。回転リングから平行な位置に円盤を設置し、中央にギアボックスモータを設置する台を設ける。90 度おきに円盤にくぼみを作り、センサーを設置できるようにした。受光部の円盤によりレーザー光が後ろに漏

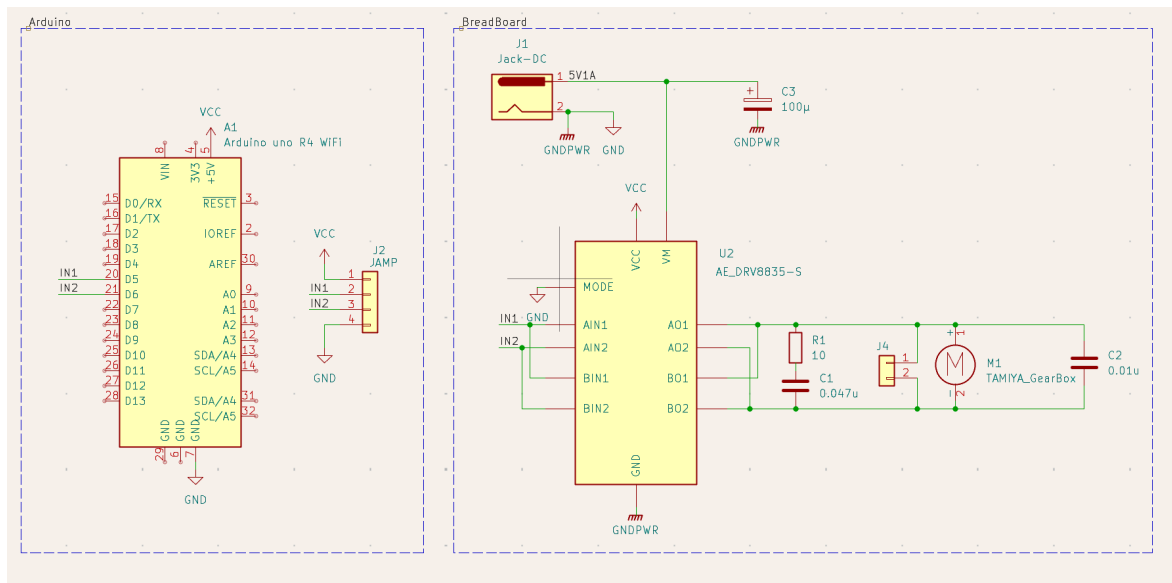


図 10 観覧車型中継機デバイス回路図

れないという設計となっている。実際に設計する部品は図 16 の通りである。

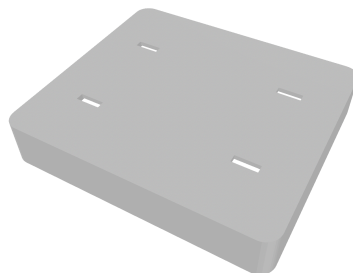


図 11 設計に使用するモデルデータ (土台部)

図～～～に実際に作成した中継機デバイスを示す。

2.2.3 内部設計

本節では中継機デバイスの内部設計について説明する。主に回転速度同期機構（モータ制御）と LEDMatrix 表示機構のプログラムについて説明する。

次に、中継機デバイスの制御プログラムのファイル構成を示す。表 10 の通り、各ファイルは機能ごとに分割されている。

FerrisWheel.ino の loop 関数は、Udp.parsePacket() でパケットの到着を監視し、届いていれば uint8_t packetBuffer[10] として 2 バイト分を読み取る。先頭バイト packetBuffer[0] をヘッダ文字として判定し、ヘッダが 'S'（演奏開始）の場合は motor.startRamp() と displayBPM() を、'B'（BPM 変更）の場合は motor.changeBPM() と displayBPM() を、'E'（演奏終了）の場合は motor.stopRamp() と

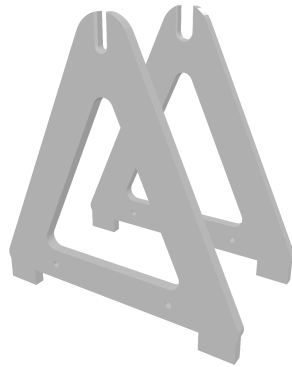


図 12 設計に使用するモデルデータ (支柱部)



図 13 設計に使用するモデルデータ (回転軸部)

clearDisplay() を、2 バイト目のペイロード（BPM 段階番号）を引数として呼び出す構成となっている。loop 関数をコード 14 に示す。

コード 14 FerrisWheel.ino の loop 関数

```

1 void loop() {
2     uint8_t packetSize = Udp.parsePacket();
3     if (packetSize <= 0) return;
4     if (packetSize != 2) {
5         while (Udp.available()) Udp.read();
6         return;
7     }
8
9     Udp.read(packetBuffer, sizeof(packetBuffer));
10    char header = packetBuffer[0];
11    uint8_t payload = packetBuffer[1];
12
13    switch (header) {
14        case 'S':
15            running = true;
16            motor.startRamp(payload);
17            displayBPM(payload);
18            break;
19        case 'B':
20            if (!running) break;
21            motor.changeBPM(payload);
22            displayBPM(payload);
23            break;

```

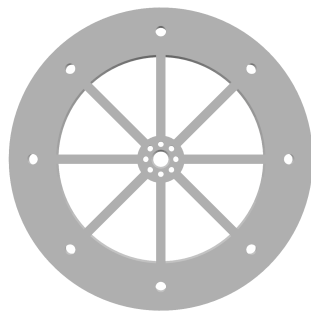


図 14 設計に使用するモデルデータ (回転リング部 1)

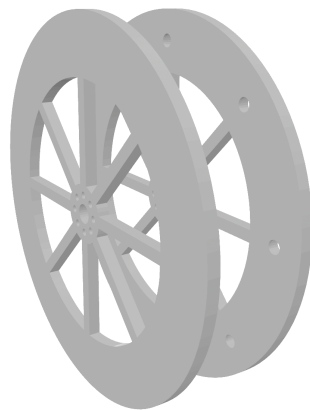


図 15 設計に使用するモデルデータ (回転リング部 2)

```

24     case 'E':
25         running = false;
26         motor.stopRamp();
27         clearDisplay();
28         break;
29     }
30 }
```

続いて、Main (FerrisWheel.ino) で使用する変数の仕様を表 11 に示す。

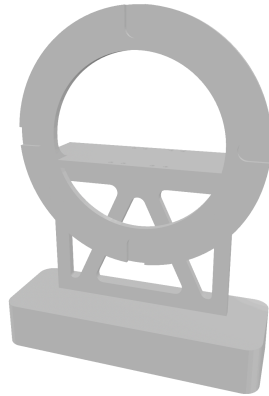


図 16 設計に使用するモデルデータ (受光部)

表 10 中継機デバイスのファイル構成

ファイル名	概要
FerrisWheel.ino	WiFi 接続・UDP 受信の管理, 受信ヘッダに応じたモータ制御・表示処理の呼び出し
Display.cpp	表示文字の独自フォント定義, LED マトリクスへの描画処理
Display.h	定数定義, 関数宣言, 外部参照
MotorControl.cpp	PWM 出力によるモータ回転制御, 回転速度テーブルの生成
MotorControl.h	モータ制御用の定数・変数定義, 関数宣言

表 11 Main (FerrisWheel.ino) で使用する変数の仕様一覧

変数名	型	説明
SSID	const char[]	Wi-Fi のネットワーク名 (hackathon001-WPA2) を定義
PASS	const char[]	その Wi-Fi のパスワード Wi-Fi パスワードを定義
localIP	IPAddress	中継機デバイス自身の静的 IP アドレスを定義
gateway	IPAddress	静的 IP 設定に使用するデフォルトゲートウェイのアドレスを定義
subnet	IPAddress	静的 IP 設定に使用するサブネットマスクを定義
LOCAL_PORT	const uint16_t	UDP 受信に利用する自身のローカルポート番号を定義
packetBuffer	uint8_t[10]	UDP 通信で受信したパケットのデータを一時的に格納するバッファ配列
running	bool	演奏中かどうかを判定し, 演奏前 (待機中) の予期せぬモータ回転を防ぐための状態管理フラグ

続いて、Main (FerrisWheel.ino) で使用する関数の仕様を表 12 に示す。

表 12 Main (FerrisWheel.ino) で使用する関数の仕様一覧

関数名	説明
loop()	UDP パケットの受信確認を行い、ヘッダが'S'であれば motor.startRamp() と displayBPM() を、'B'であれば motor.changeBPM() と displayBPM() を、'E'であれば motor.stopRamp() と clearDisplay() を呼び出す。演奏前 (running=false) に'B'を受信した場合は無視する。

ここからは、モータの制御、LEDMatrix での BPM 表示、回転速度実測プログラム (LaserIntervalTest) の順に、使用する変数・関数の仕様と処理内容について説明する。

まず、中継機デバイスにおけるモータの制御処理について説明する。本デバイスのギアボックスモータの回転速度を制御するために、Arduino の PWM(パルス幅変調) 機能を用いてモータの平均印加電圧を制御する。PWM とは、デジタル出力ピンから出力される矩形波の HIGH 状態と LOW 状態の時間比率(デューティ比)を変化させることで、擬似的にアナログ電圧を生成する制御方式である。Arduino のデジタル出力ピンは 5V の HIGH か 0V の LOW のいずれかしか出力できないが、HIGH と LOW を高速に切り替えてデューティ比を調整することで、負荷に対して見かけ上の連続的な電圧を印加することが可能となる。

本デバイスは、Arduino の D5 ピンをモータドライバモジュールに接続し、PWM 制御を行う構成としている。Arduino の analogWrite 関数は、引数として 0 から 255 の整数値を受け取り、この値に応じたデューティ比の矩形波を出力する。このとき引数が 0 のときデューティ比は常時 LOW であり、255 のときデューティ比は常時 HIGH となる。中間値では、引数に比例したデューティ比の矩形波が生成される。この関係を式で表すと、供給電圧を V_{cc} (定数 VCC)、プログラム上の出力値を $pwmValue$ とすると、モータに加わる平均電圧 $V_{average}$ は (1) 式で決定される。

$$V_{average} = V_{cc} \times \frac{pwmValue}{255} \quad (1)$$

このとき、モータの巻線が持つインダクタンスとギアボックスの機械的慣性が電氣的・機械的なローパスフィルタとして機能するため、モータはパルスの平均値に応じた一定速度で滑らかに回転する。

次に、ルックアップテーブルによる速度制御について説明する。一般的な DC モータの回転速度は供給電圧に比例する性質があるが [11]、実際には摩擦や機構の負荷などによる非線形な挙動を示す事が多い。そのため、当初検討していた「4 拍で 90 度 (1 小節が 90 度) 回転する」といった幾何学的な理論式による制御ではなく、より確実な速度追従を実現するため、幾何学的な縛りを廃止して実測値に基づくルックアップテーブル方式を採用する。

本デバイスでは、表 13 に示す離散的な 5 段階の BPM 設定値と、あらかじめ実測により求めた PWM 出力値を、コード内の配列 bpmTable・pwmTable として対応させ、受信した段階番号に応じた pwmValue を即座に選択・出力する構成とする。表 13 に示す実測周期は、後述する回転速度実測プログラム（LaserIntervalTest）を用いて計測した値に基づいて決定している。

表 13 BPM に対する実測周期と出力 PWM 値			
段階	目標 BPM	実測周期 [ms]	出力 PWM 値
1	60	230	26
2	90	180	28
3	120	140	30
4	150	110	32
5	180	80	37

続いて、モータ制御（MotorControl クラス）で使用する変数・定数の仕様を表 14 に示す。

表 14 モータ制御（MotorControl クラス）で使用する変数・定数の仕様一覧		
変数名	型	説明
PIN_MOTOR_PWM	const uint8_t	モータ制御用の出力ピンを定義
bpmTable[5]	const uint8_t	5 つの離散値 BPM を格納
pwmTable[5]	const uint8_t	5 つの離散値 PWM 出力値を格納
VCC	const float	Arduino への供給電圧 V_{cc} を定義
currentBPM	uint8_t	現在演奏中の BPM データを格納
RPM	float	BPM から計算された目標回転数を格納
omega	float	目標角速度 ω を格納
pwmValue	uint8_t	analogWrite 用の出力値を格納
V_average	float	平均電圧 $V_{average}$ を格納
rpmTable[5]	float	bpmTable に対応する目標回転数を格納する配列

続いて、モータ制御（MotorControl クラス）で使用する関数の仕様を表 15 に示す。

表 15 モータ制御（MotorControl クラス）で使用する関数の仕様一覧

関数名	説明
begin()	setup 内で一度だけ呼び出される関数であり，モータピンの初期化を行う
createRpmTable()	bpmTable の各値から目標回転数を算出し，rpmTable を作成する
startRamp(stepNumber)	演奏開始時の加速を制御する．段階番号から求めた目標の pwmValue まで，PWM 出力値を 5 ずつ段階的に上げていくことで滑らかに目標速度へ到達させる
stopRamp()	演奏終了時の減速を制御する．現在の pwmValue から 0 まで，PWM 出力値を 5 ずつ段階的に下げていくことで滑らかに停止させる
changeBPM(stepNumber)	演奏中の BPM 変更を行う．startRamp とは異なり，段階的な加減速を行わず目標の pwmValue を即座に出力する
stepToNumber(stepNumber)	受信した段階番号 (1～5) から，bpmTable・pwmTable・rpmTable を参照して currentBPM・pwmValue・RPM を求める
updateStatus()	RPM から角速度 omega を，pwmValue から平均電圧 V_average を算出する

以下では，表 15 に示した順に，各関数の処理内容を実際のコードとともに説明する．はじめに，begin 関数について説明する．この関数は，setup 内で一度だけ呼び出され，モータドライバへ接続された PIN_MOTOR_PWM を出力ピンとして初期化したうえで，出力を 0 にリセットする．begin 関数をコード 15 に示す．

コード 15 MotorControl::begin 関数

```

1 void MotorControl::begin() {
2     pinMode(PIN_MOTOR_PWM, OUTPUT);
3     analogWrite(PIN_MOTOR_PWM, 0);
4 }
```

次に，createRpmTable 関数について説明する．この関数も setup 内で一度だけ呼び出され，bpmTable の各要素を 16 で除算することで rpmTable を生成する．createRpmTable 関数をコード 16 に示す．

コード 16 MotorControl::createRpmTable 関数

```

1 void MotorControl::createRpmTable() {
2     for (uint8_t i = 0; i < 5; i++) {
3         rpmTable[i] = bpmTable[i] / 16.0f;
4     }
5 }
```

次に、startRamp 関数について説明する。この関数は、まず stepToNumber 関数によって目標の pwmValue を求めたうえで、出力値を 0 から pwmValue まで 5 刻みで増加させながら 40 ms ずつ待機することで、モータを滑らかに加速させる。startRamp 関数をコード 17 に示す。

コード 17 MotorControl::startRamp 関数

```
1 void MotorControl::startRamp(uint8_t stepNumber) {
2     stepToNumber(stepNumber);
3     for (int i = 0; i <= pwmValue; i += 5) {
4         analogWrite(PIN_MOTOR_PWM, i);
5         delay(40);
6     }
7     analogWrite(PIN_MOTOR_PWM, pwmValue);
8 }
```

次に、stopRamp 関数について説明する。この関数は startRamp 関数と対になる処理であり、現在の pwmValue から 0 まで 5 刻みで減少させながら 40 ms ずつ待機することで、モータを滑らかに減速・停止させる。stopRamp 関数をコード 18 に示す。

コード 18 MotorControl::stopRamp 関数

```
1 void MotorControl::stopRamp() {
2     for (int i = pwmValue; i >= 0; i -= 5) {
3         analogWrite(PIN_MOTOR_PWM, i);
4         delay(40);
5     }
6     analogWrite(PIN_MOTOR_PWM, 0);
7 }
```

次に、changeBPM 関数について説明する。この関数は演奏中に BPM の段階が変更された際に呼び出され、stepToNumber 関数と updateStatus 関数によって新しい段階の値を求めたうえで、startRamp 関数のような段階的な加減速を行わず、目標の pwmValue を即座に出力する。演奏を止めずにテンポを追従させるため、段階的な加減速は行わない仕様としている。changeBPM 関数をコード 19 に示す。

コード 19 MotorControl::changeBPM 関数

```
1 void MotorControl::changeBPM(uint8_t stepNumber) {
2     stepToNumber(stepNumber);
3     updateStatus();
4     analogWrite(PIN_MOTOR_PWM, pwmValue);
5 }
```

最後に、stepToNumber 関数と updateStatus 関数について説明する。stepToNumber 関数は、受信した段階番号(1~5)が範囲内であることを確認したうえで、bpmTable・pwmTable・rpmTable の該当インデックスから currentBPM・pwmValue・RPM をそれぞれ求める。updateStatus 関数は、この RPM および pwmValue を基に、(1) 式に相当する計算によって omega と V_average を算出する。両関数をコード 20 に示す。

コード 20 MotorControl::stepToNumber 関数・updateStatus 関数

```

1 void MotorControl::stepToNumber(uint8_t stepNumber) {
2     if (stepNumber < 1 || stepNumber > 5) return;
3     currentBPM = bpmTable[stepNumber - 1];
4     pwmValue   = pwmTable[stepNumber - 1];
5     RPM        = rpmTable[stepNumber - 1];
6 }
7
8 void MotorControl::updateStatus() {
9     omega      = 2.0 * PI * RPM / 60.0f;
10    V_average = VCC * (pwmValue / 256.0f);
11 }

```

次に、中継機デバイスにおける LEDMatrix 表示処理について説明する。今回のシステムでは表示用のマトリクスとして、Arduino Uno R4 WiFi 付属のマトリクスを使用する。このマトリクスを用いて、FerrisWheel.ino の loop 関数から受信した BPM 段階番号を数値変換し、独自定義したフォント配列を用いて表示を行う。ここで、今回利用する Arduino のマトリクスは縦 12 横 8 の範囲で構成されている。この時、通常で定義されている文字列では文字の間隔が短くなることや、それに付随した可読性の低下が考えられる。よって、今回は 1 文字を縦 5 横 3 で再定義することにより文字同士の間隔を開け、可読性の担保を目指す。例として数字 0 のドットパターンをコード 21 に示す。

コード 21 font3x5 配列における数字 0 のドットパターン定義

```

1 {1,1,1,
2   1,0,1,
3   1,0,1,
4   1,0,1,
5   1,1,1}

```

また、LED マトリクスの描画処理は、受信した段階番号から BPM 値を参照し、整数を各桁に分割した後、独自定義した 3 × 5 のフォント配列を用いてフレームバッファに書き込むことで行う。続いて、LEDMatrix 表示 (Display モジュール) で使用する変数・定数の仕様を表 16 に示す。

表 16 LEDMatrix 表示 (Display モジュール) で使用する変数・定数の仕様一覧

変数名	型	説明
BPM_STEP_1～5	#define	BPM5 段階 (60,90,120,150,180) の各値を定義
FONT_WIDTH	#define	1 文字の横ドット数 (3) を定義
FONT_HEIGHT	#define	1 文字の縦ドット数 (5) を定義
MATRIX_ROWS	#define	LED マトリクスの行数 (8) を定義
MATRIX_COLS	#define	LED マトリクスの列数 (12) を定義
font3x5[10][15]	const uint8_t	独自定義した 0～9 の 3 × 5 ドットパターンフォント配列
matrix	ArduinoLEDMatrix	LED マトリクスのインスタンス
bpmTable[5]	static const int	段階番号 (1～5) から実際の BPM 値へ変換するための対応表
frameBuffer[8][12]	static uint8_t	描画用の共有フレームバッファ
currentBPM	static int	現在表示中の BPM 値を保持する内部状態変数。clearDisplay 呼び出し時に初期値 (-1) へリセットされる

続いて、LEDMatrix 表示 (Display モジュール) で使用する関数の仕様を表 17 に示す。

表 17 LEDMatrix 表示 (Display モジュール) で使用する関数の仕様一覧

関数名	説明
displaySetup()	LED マトリクスの初期化を行う
displayBPM(stepNumber)	受信した BPM 段階番号を LED マトリクスに数値表示する
drawDigit(digit, x_offset)	1 桁の数字をフレームバッファの指定位置に描画する
clearDisplay()	LED マトリクスを全消灯する

LEDMatrix プログラムのクラス図は以下の図 17 の通りである。WiFi 接続・UDP 受信および各処理の呼び出しを行う FerrisWheel.ino、LED マトリクスへの描画処理を行う Display.cpp、および関数宣言やフォント定義を行う Display.h で構成される。各モジュール間の関係について、FerrisWheel.ino から Display.cpp への関係は、表示処理を利用する依存関係である。また、Display.cpp から Display.h への関係は、関数宣言およびフォント定義を参照する依存関係である。

以下では、表 17 に示した順に、各関数の処理内容を実際のコードとともに説明する。はじめに、displaySetup 関数について説明する。この関数は、matrix.begin() を呼び出した後、消灯フレームを一度描画してから 200 ms 待機する。これは、begin() 直後の最初の loadFrame() は反映されない場合があるための対策である。displaySetup 関数をコード 22 に示す。

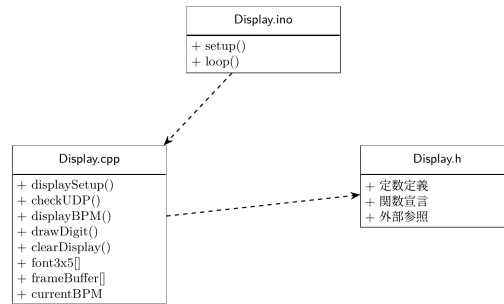


図 17 LEDMatrix のクラス図

コード 22 displaySetup 関数

```

1 void displaySetup() {
2     matrix.begin();
3     uint32_t warmUp[3] = {0, 0, 0};
4     matrix.loadFrame(warmUp);
5     delay(200);
6 }
  
```

次に、displayBPM 関数について説明する。この関数は、loop 関数から受け取った BPM 段階番号（1～5）を bpmTable[] で参照して実際の BPM 値に変換したうえで各桁ごとに分割し、drawDigit() を用いてフレームバッファに描画する。そして、uint32_t[3] 形式にビットパックした後、matrix.loadFrame() で描画を行う。displayBPM 関数をコード 23 に示す。

コード 23 displayBPM 関数

```

1 void displayBPM(uint8_t stepNumber) {
2     if (stepNumber < 1 || stepNumber > 5) return;
3     uint8_t bpm = bpmTable[stepNumber - 1];
4
5     memset(frameBuffer, 0, sizeof(frameBuffer));
6
7     int digits[3];
8     int numDigits;
9     if (bpm >= 100) {
10         numDigits = 3;
11         digits[0] = bpm / 100;
12         digits[1] = (bpm / 10) % 10;
13         digits[2] = bpm % 10;
14     } else if (bpm >= 10) {
15         numDigits = 2;
16         digits[0] = bpm / 10;
17         digits[1] = bpm % 10;
18     } else {
19         numDigits = 1;
20         digits[0] = bpm;
21     }
22 }
  
```

```

23     int totalWidth = numDigits * FONT_WIDTH + (numDigits - 1) * 1;
24     int startX     = (MATRIX_COLS - totalWidth) / 2;
25
26     for (int i = 0; i < numDigits; i++) {
27         drawDigit(digits[i], startX + i * (FONT_WIDTH + 1));
28     }
29
30     uint32_t bitmap[3] = {0, 0, 0};
31     for (int row = 0; row < MATRIX_ROWS; row++) {
32         for (int col = 0; col < MATRIX_COLS; col++) {
33             if (frameBuffer[row][col]) {
34                 int n = row * MATRIX_COLS + col;
35                 bitmap[n / 32] |= (1UL << (31 - (n % 32)));
36             }
37         }
38     }
39     matrix.loadFrame(bitmap);
40 }

```

次に、drawDigit 関数について説明する。この関数は、引数の数字が 0 から 9 の範囲内であるかを確認した後、縦方向の中央寄せオフセットを算出し、独自フォント配列から該当する数字のドットパターンを 1 ドットずつ読み取ってフレームバッファの指定位置に書き込む。drawDigit 関数をコード 24 に示す。

コード 24 drawDigit 関数

```

1 void drawDigit(int digit, int x_offset) {
2     if (digit < 0 || digit > 9) return;
3     const int y_offset = (MATRIX_ROWS - FONT_HEIGHT) / 2;
4     for (int y = 0; y < FONT_HEIGHT; y++) {
5         for (int x = 0; x < FONT_WIDTH; x++) {
6             int col = x_offset + x;
7             int row = y_offset + y;
8             if (col >= 0 && col < MATRIX_COLS && row >= 0 && row < MATRIX_ROWS) {
9                 frameBuffer[row][col] = font3x5[digit][y * FONT_WIDTH + x];
10            }
11        }
12    }
13 }

```

最後に、clearDisplay 関数について説明する。この関数は、BPM 値を初期値 (-1) にリセットし、全消灯フレームを matrix.loadFrame() で描画して表示のリセットを行う。clearDisplay 関数をコード 25 に示す。

コード 25 clearDisplay 関数

```

1 void clearDisplay() {
2     currentBPM = -1;
3     uint32_t blank[3] = {0, 0, 0};
4     matrix.loadFrame(blank);

```

最後に、中継機デバイスのモータ回転速度を実測するために用いたプログラムである `LaserIntervalTest` について説明する。このプログラムは中継機デバイス本体には搭載せず、開発段階で観覧車に設置したレーザーと CdS セルを用いて、各 PWM 出力値における実際の回転周期を計測するために使用したものであり、表 13 に示した実測周期の値はこのプログラムによって得られている。本プログラムは、CdS セルの出力値を監視し、レーザー光の通過（パルス）を検知するたびに、直前のパルス検知からの経過時間（interval）をシリアルモニタへ出力する。周囲光量やレーザーの取り付け誤差による検知レベルのばらつきに対応するため、固定閾値ではなく、起動直後のキャリブレーションと直近のピーク値履歴に基づいて検知しきい値を継続的に調整する適応しきい値方式を採用している。`LaserIntervalTest` で使用する主な定数の仕様を表 18 に示す。

表 18 `LaserIntervalTest` で使用する主な定数の仕様一覧

定数名	型		説明
<code>CDS_PIN</code>	<code>const int</code>		CdS セルの出力を読み取るアナログピン番号を定義
<code>CDS_ON_THRESHOLD</code> <code>CDS_OFF_THRESHOLD</code>	/	<code>const int</code>	起動直後にのみ使用する仮の ON/OFF しきい値を定義
<code>CDS_MIN_DELTA</code>	<code>const int</code>		暗い状態との差がこの値未満であればレーザーとして扱わない閾値を定義
<code>CDS_PEAK_HISTORY</code>	<code>const uint8_t</code>		しきい値の再計算に用いる直近ピーク値の保持個数を定義
<code>CDS_ON_RATIO</code> / <code>CDS_OFF_RATIO</code>	<code>const uint8_t</code>		baseline からピーク値までの何%を ON/OFF しきい値とするかを定義
<code>CDS_THRESHOLD_STEP_LIMIT</code>	<code>const int</code>		1 回の再計算でしきい値が変化できる上限幅を定義
<code>CDS_STARTUP_CALIBRATION_MS</code>	<code>const long</code>	<code>unsigned</code>	起動直後の誤検出を防止するキャリブレーション時間を定義
<code>DEBOUNCE_MS</code>	<code>const long</code>	<code>unsigned</code>	同一パルスの多重検知を防止するデバウンス時間を定義

続いて、`LaserIntervalTest` (`LaserDetector` クラス) で使用する主な関数の仕様を表 19 に示す。

表 19 LaserIntervalTest (LaserDetector クラス) で使用する主な関数の仕様一覧

関数名	説明
update(cdsValue, nowMs, intervalOut)	loop 内で毎回呼び出される公開関数. detectLight による検知およびデバウンス処理を経て, 新規パルスを検知した場合に true を返し, 前回検知からの経過時間 intervalOut を書き込む
detectLight(v, nowMs)	baseline の追従, 起動直後のキャリブレーション, ヒステリシスを持った ON/OFF 判定によってレーザーパルスの立ち上がりを検知する内部関数
registerPeak(peak)	検知したパルスのピーク値を履歴に登録し, その平均値から ON/OFF しきい値を再計算する内部関数
updateFallbackThresholds()	baseline の値から ON/OFF しきい値の暫定値を算出する内部関数
limitStep(current, target, limit)	しきい値の 1 回あたりの変化量を limit 以内に制限する内部関数

以下では, 表 19 に示した主要な関数のうち, 公開インタフェースである update 関数と, 検知処理の中心である detectLight 関数について, 実際のコードとともに説明する. なお, registerPeak 関数・updateFallbackThresholds 関数・limitStep 関数の詳細な実装は付録に示すプログラム全文を参照されたい.

はじめに, setup 関数と loop 関数について説明する. setup 関数ではシリアル通信を開始するのみであり, loop 関数では CdS セルの値を毎回取得し, LaserDetector の update 関数へ渡している. 新しいパルスが検知された場合は, 得られた interval[ms] をシリアルモニタへ出力する. setup 関数・loop 関数をコード 26 に示す.

コード 26 LaserIntervalTest.ino の setup 関数・loop 関数

```

1 void setup() {
2   Serial.begin(115200);
3   Serial.println("---_レーザー間隔計測_Setup_Started_---");
4 }
5
6 void loop() {
7   unsigned long now      = millis();
8   int           cdsValue = analogRead(CDS_PIN);
9   unsigned long interval = 0;
10
11   if (detector.update(cdsValue, now, interval)) {
12     Serial.print("[interval]_");
13     Serial.print(interval);
14     Serial.println("_ms");
15   }

```

16 }

次に、LaserDetector クラスの update 関数について説明する。この関数は、detectLight 関数によってパルスの立ち上がりが検知され、かつ前回検知から DEBOUNCE_MS (200 ms) 以上が経過している場合にのみ、true を返して interval (前回検知時刻との差) を書き込む。update 関数をコード 27 に示す。

コード 27 LaserDetector::update 関数

```
1 bool update(int cdsValue, unsigned long nowMs, unsigned long &intervalOut) {
2     if (calibrationStartMs == 0) calibrationStartMs = nowMs;
3     if (!detectLight(cdsValue, nowMs)) return false;
4     if (nowMs - lastDetectTime < DEBOUNCE_MS) return false;
5
6     intervalOut = (lastDetectTime != 0) ? (nowMs - lastDetectTime) : 0;
7     bool hasInterval = (lastDetectTime != 0);
8     lastDetectTime = nowMs;
9     return hasInterval;
10 }
```

最後に、detectLight 関数について説明する。この関数は、暗状態の推定値である baseline を継続的に追従させながら、値が baseline+CDS_MIN_DELTA 相当の onThreshold を上回った瞬間をパルスの立ち上がりとして検知する状態機械である。起動直後は startupCalibrated フラグが false の間、CDS_STARTUP_CALIBRATION_MS (1 秒) をかけて最も暗い値を baseline の初期値として探索し、誤検知を防止する。また、キャリブレーション完了時点でレーザーが既に照射状態であった場合は、waitingForRelease フラグによって一度非照射状態に戻るまで検知を保留する。パルスの立ち下がり (照射終了) を検知すると registerPeak 関数を呼び出し、しきい値を実測値に基づいて更新する。detectLight 関数をコード 28 に示す。

コード 28 LaserDetector::detectLight 関数

```
1 bool detectLight(int v, unsigned long nowMs) {
2     if (!baselineReady) {
3         baseline = v;
4         startupMin = v;
5         baselineReady = true;
6         updateFallbackThresholds();
7     }
8
9     if (!startupCalibrated) {
10         if (v < startupMin) startupMin = v;
11         baseline = startupMin;
12         updateFallbackThresholds();
13         if (nowMs - calibrationStartMs < CDS_STARTUP_CALIBRATION_MS) return false;
14         startupCalibrated = true;
15         waitingForRelease = (v >= onThreshold);
16         releaseThreshold = (baseline > 1023 - CDS_MIN_DELTA) ? baseline - CDS_MIN_DELTA
17             : offThreshold;
18         return false;
19     }
20 }
```

```

18     }
19
20     if (waitingForRelease) {
21         if (v <= releaseThreshold) {
22             baseline = v;
23             updateFallbackThresholds();
24             waitingForRelease = false;
25         }
26         return false;
27     }
28
29     if (!laserOn) {
30         if (v < baseline) baseline = v;
31         else baseline = (baseline * 31 + v) / 32;
32         if (!calibrated) updateFallbackThresholds();
33     }
34
35     if (!laserOn && v >= onThreshold) {
36         laserOn = true;
37         pulsePeak = v;
38         return true;
39     }
40
41     if (laserOn) {
42         if (v > pulsePeak) pulsePeak = v;
43         if (v <= offThreshold) {
44             laserOn = false;
45             registerPeak(pulsePeak);
46         }
47     }
48     return false;
49 }

```

プログラム全文は付録に示す。

2.3 楽器デバイス

果たしてデバイスなのか_____

3 システム実装

本節では、担当したハードウェアの詳細設計と LEDMatrix での BPM 表示を行うプログラムについて説明する。

3.1 部品一覧

次に、楽器デバイスが受光するための部品について表 20 に示す。光センサモジュールとして CdS セルを使用する。CdS セルにレーザー光が照射された際の出力を Arduino のアナログピンへ接続することで、観覧車デバイスのレーザー光の通過を検知する。また、今回はドラム、ピアノ、ストリングス、フルートの 4 種類の楽器の音色を再現するためそれぞれに Arduino を使用し、光センサおよびブレッドボードも 4 台用意する。

表 20 楽器デバイスに必要な機器

機材	品番	個数 [個]
Arduino UNO R4	-	4
CdS セル	GL5516	4
ブレッドボード	-	4
ジャンパーワイヤー	-	適宜

3.2 指揮デバイス

3.3 観覧車型中継機デバイス

本デバイスのモータ回転制御および LED マトリクス表示を行う制御プログラムの全文は付録 (FerrisWheel.ino, Display.h, Display.cpp, MotorControl.h, MotorControl.cpp) に示す。

3.4 レーザー光送信装置

レーザー光送信装置の詳細設計について説明する。図 18 について、回路図の通りにボタン電池とレーザーをはんだ付けする。これらを観覧車のリングに 45 度間隔に装着する。電源としては図 18 に示すようにボタン電池からの 3V 給電を行う。ここで、レーザーについて、データシートより定電流源回路を組み込んであるため、現在の設計であれば外部抵抗は必要とせず、素子を安全に利用することができる [?].

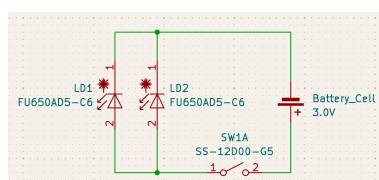


図 18 レーザー回路図

3.5 楽器デバイス受光部装置

楽器デバイスの受光部の回路図を図 19 に示す。電源仕様は、PC との USB 接続から電力を供給する。光センサは Arduino の 5V ピンから給電されるため、外部電源は不要である。また、光センサが光を受光した瞬間に A0 のアナログピンにセンサ値を出力する。このセンサ値の変化を Arduino で処理することで光を検知する仕様である。

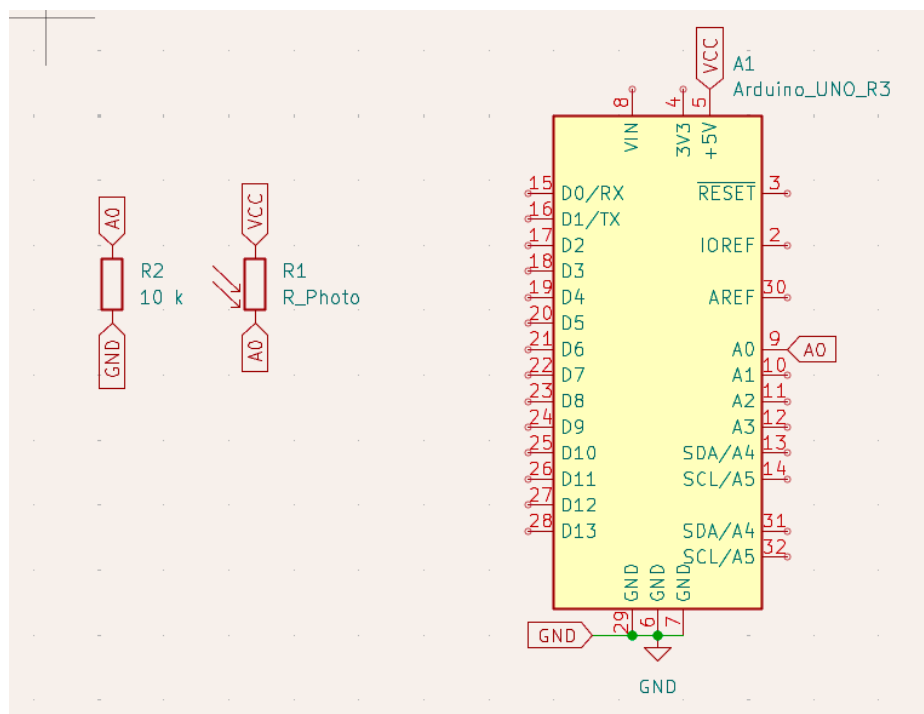


図 19 楽器デバイスの受光部の回路図

4 おわりに

これは $\text{T}_\text{E}\text{X}$ による報告書作成のテンプレートでした.

参考文献

- [1] 株式会社マーケットリサーチセンター, “音楽出版の日本市場(～2031年)、市場規模(パフォーマンス、同期、デジタル収益)・分析レポートを発表,” <https://www.atpress.ne.jp/news/4436457>, 2026/4/22 参照.
- [2] LP Information, “ライブコンサート市場占有率分析レポート,” <https://www.atpress.ne.jp/news/3304316>, 2026/4/22 参照.
- [3] 一般社団法人コンサートプロモーターズ協会, “ライブ市場調査 基礎推移表” <https://www.acpc.or.jp/marketing/transition/>, 2026/5/18 参照.
- [4] 谷口由貴, “ストーリーミングサービスが生み出した新たな音楽消費サイクル,” 博報堂 The Hakuhodo, <https://www.hakuhodo.co.jp/magazine/61505/>, 2019/9/12, 2026/7/7 参照.
- [5] Martin Kreuzer, Melanie Wald-Fuhrmann, Christian Weining, Martin Tröndle, and Hauke Egermann, “Western Classical Music Concerts Are More Immersive, Intellectually Stimulating, and Social, When Experienced Live Rather Than in a Digital Stream: An Ecologically Valid Concert Study on Different Modes of Liveness,” *Music & Science*, vol.8, 2025.
- [6] Natalia Cooper, Dimitrios Mileounis, Patrick Williams, and Frank P. Brooks, “The effects of substitute multisensory feedback on task performance and the sense of presence in a virtual reality environment,” *PLOS ONE*, vol.13, no.2, p.e0191846, 2018.
- [7] Martin, R., and Nielsen, N. Enacting Musical Aesthetics: The Embodied Experience of Live Music. *Music & Science*, 7, 2024
- [8] Yamaha Music Japan, “ENSPIRE PRO,” https://jp.yamaha.com/products/musical_instruments/pianos/disklavier/enspire_pro/features.html#product-tabs, 2026/5/19 参照.
- [9] 関根 伸也, “DMX512 信号システムなどの現状技術,” 電気設備学会誌, vol.41, no.7, 2026, p.382-385, 2021
- [10] ReacTable Legacy, “Welcome|ReacTable Legacy,” <https://reactable.com/>, 2026/5/19 参照.
- [11] Control.com, “DC Motor Speed Control,” *Lessons In Electric Circuits*, <https://control.com/textbook/variable-speed-motor-controls/dc-motor-speed-control/>, (参照 2026-05-04).

A 役割分担とスケジュール

このテンプレートでは、 $\text{T}_\text{E}\text{X}$ でガントチャートを描いてみました。

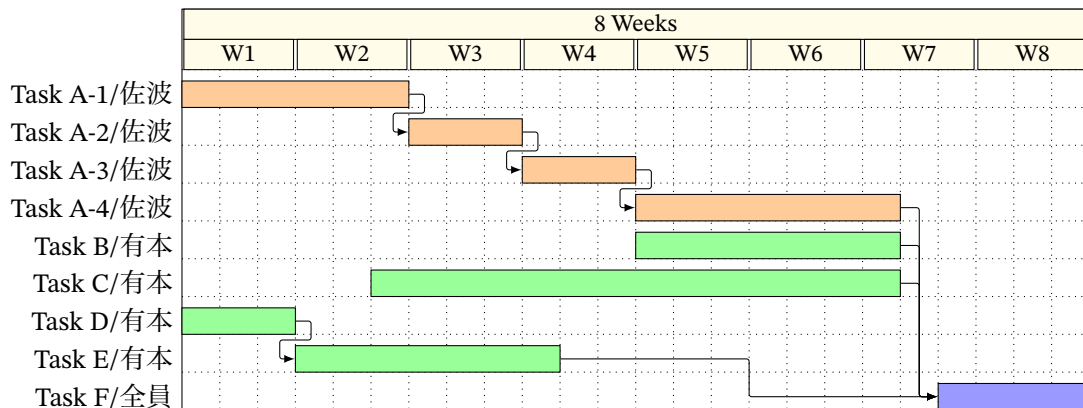


図 A.20 計画時のスケジュール

B 指揮デバイスのプログラムソース

本節では、指揮デバイスの制御プログラムの全文を示す。ファイル構成は 2.1.3 節の表 2 の通りである。

B.1 Controller_simultaneous.ino

コード 29 Controller_simultaneous.ino

```

1  #include "SyncManager.h"
2  #include "PotManager.h"
3  #include "AccManager.h"
4
5  #include <Wire.h>
6  #include <math.h>
7
8  // Wi-Fi接続情報
9  const char SSID[] = "hackathon001-WPA2";
10 const char PASS[] = "hackathon001";
11
12 const uint8_t input_PIN = 3; // スイッチの状態を読み取るピン
13 bool switch_status;
14 bool before_switch_status = 0;
15 bool isButtonPlaying = false;
16
17 SyncManager sync;
18 PotManager pot;
19 AccManager acc;
20
21 void setup() {
22     Serial.begin(115200);
23     while (!Serial) {}

```



```

24
25     Serial.println("---_Setup_Started_---");
26     pinMode(input_PIN, INPUT);
27
28     Serial.println("Attempting_to_connect_to_WiFi...");
29     sync.begin(SSID, PASS);
30
31     Serial.println("\nWiFi_Connected_successfully!");
32     Serial.println("Commands:_button_->_Start/End");
33
34     acc.setupADXL345();
35
36     before_switch_status = digitalRead(input_PIN);
37 }
38
39 void loop() {
40     button();
41     pot.update();
42     acc.update(sync, pot);
43 }
44
45 void button() {
46     switch_status = digitalRead(input_PIN);
47     if (switch_status == 0 && before_switch_status == 1) {
48         if (isButtonPlaying) {
49             Serial.println("Button:_[END]_sending_UDP...");
50             sync.sendEnd();
51             isButtonPlaying = false;
52         } else {
53             Serial.println("Button:_[START]_sending_UDP_to_all_simultaneously...");
54             uint8_t step = pot.getBpmStep();
55             sync.sendStart(step);
56             isButtonPlaying = true;
57         }
58         delay(300); // チャタリング防止
59     }
60     before_switch_status = switch_status;
61 }

```

B.2 AccManager.h

コード 30 AccManager.h

```

1 // 加速度センサー管理クラス
2 #ifndef ACC_MANAGER_H
3 #define ACC_MANAGER_H
4
5 #include <Arduino.h>

```

```

6  #include <Wire.h>
7  #include "SyncManager.h"
8  #include "PotManager.h"
9
10 class AccManager {
11 public:
12     AccManager();
13     void setupADXL345();
14     // loop内で毎ループ呼ぶ
15     void update(SyncManager& sync, PotManager& pot);
16
17     void updateShakedInfo(SyncManager& sync, PotManager& pot);
18
19 private:
20     float readAccMagnitude();
21     void recoverI2C();    // I2Cバスロックアップからの復旧（バスクリア+再初期化）
22
23     const uint8_t ADXL345_ADDR = 0x53; // SDO/ALT ADDRESS ピンがGNDの場合
24     const uint8_t REG_POWER_CTL = 0x2D; // 加速度センサの電源管理をするスイッチ
25     const uint8_t REG_DATA_FORMAT = 0x31; // 測定する範囲を設定するスイッチ
26     const uint8_t REG_DATAX0 = 0x32; // 測定した値が書き込まれる場所
27
28     const float accThreshold = 110.0f; // 検知閾値 必要に応じて調整
29     const unsigned long shakeInterval = 750; // 多重検知防止インターバル [ms]
30     const unsigned long ACC_SAMPLE_INTERVAL = 20;
31
32     // 内部変数
33     float currentAccVal = 0.0f; // 現在の加速度センサ値
34     float lastAccVal = 0.0f; // 前回の加速度センサ値
35     unsigned long lastDetectTime = 0; // 前回検知時刻 [ms]
36     unsigned long lastSampleTime = 0;
37     unsigned long lastRecoverMs = 0; // 復旧ログのスロットル用
38
39     bool isStarted = false; // 振られた判定のフラグ
40 };
41
42 #endif

```

B.3 AccManager.cpp

コード 31 AccManager.cpp

```

1  #include "AccManager.h"
2
3  AccManager::AccManager(){
4      // コンストラクタ（初期化処理なし）
5  }
6

```

```

7 void AccManager::setupADXL345() {
8     Wire.begin();
9
10    Wire.beginTransmission(ADXL345_ADDR);
11    Wire.write(REG_POWER_CTL);
12    Wire.write(0x08); // Measure bit ON
13    Wire.endTransmission();
14
15    Wire.beginTransmission(ADXL345_ADDR);
16    Wire.write(REG_DATA_FORMAT);
17    Wire.write(0x00);
18    Wire.endTransmission();
19
20    lastAccVal = readAccMagnitude();
21
22    Serial.println("GY-291_初期化完了");
23    Serial.print("閾値:");
24    Serial.print(accThreshold);
25    Serial.print("インターバル:");
26    Serial.print(shakeInterval);
27    Serial.println("_ms");
28 }
29
30 // ★loopから引っ越してきた時間管理とシェイク判定
31 void AccManager::update(SyncManager& sync, PotManager& pot) {
32     unsigned long now = millis();
33
34     if (now - lastSampleTime >= ACC_SAMPLE_INTERVAL) {
35         lastSampleTime = now;
36
37         currentAccVal = readAccMagnitude();
38         float diff = fabsf(currentAccVal - lastAccVal);
39
40         bool overThreshold = (diff > accThreshold);
41         bool intervalPassed = ((now - lastDetectTime) >= shakeInterval);
42
43         // 500msごとにdiff値を表示 (閾値チューニング用)
44         static unsigned long lastDebugTime = 0;
45         if (now - lastDebugTime >= 500) {
46             lastDebugTime = now;
47             //Serial.print("[ACC] diff="); Serial.print(diff, 1);
48             //Serial.print(" / 閾値="); Serial.println(accThreshold, 1);
49         }
50
51         if (overThreshold && intervalPassed) {
52             lastDetectTime = now;
53             isStarted = !isStarted;
54

```

```

55     Serial.print("[SHAKE検知]_diff=");
56     Serial.print(diff, 4);
57     Serial.println(isStarted ? "_START送信" : "_END送信");
58
59     updateShakedInfo(sync, pot);
60 }
61
62     lastAccVal = currentAccVal;
63 }
64 }
65
66 float AccManager::readAccMagnitude() {
67     Wire.beginTransaction(ADXL345_ADDR);
68     Wire.write(REG_DATA_X0);
69     // endTransmission!=0 はNACK/バス異常。リピーテッドスタート前に検出する。
70     if (Wire.endTransmission(false) != 0) {
71         recoverI2C();
72         return lastAccVal;    // 直前値を返してニセshakeを防ぐ
73     }
74
75     uint8_t n = Wire.requestFrom(ADXL345_ADDR, (uint8_t)6, (uint8_t)true);
76     if (n < 6) {                // 6バイト読めない=ロックアップ等
77         recoverI2C();
78         return lastAccVal;
79     }
80
81     int16_t rawX = (int16_t)((Wire.read() | (Wire.read() << 8)));
82     int16_t rawY = (int16_t)((Wire.read() | (Wire.read() << 8)));
83     int16_t rawZ = (int16_t)((Wire.read() | (Wire.read() << 8)));
84
85     float fx = (float)rawX;
86     float fy = (float)rawY;
87     float fz = (float)rawZ;
88
89     return sqrtf(fx * fx + fy * fy + fz * fz);
90 }
91
92 // I2Cバスのロックアップから復旧する。
93 // スレーブがSDAを握ったまま固まった場合、SCLを手で9回叩いて解放し、
94 // STOP条件を作ってから Wire を張り直し、ADXL345 を再初期化する。
95 void AccManager::recoverI2C() {
96     unsigned long now = millis();
97     if (now - lastRecoverMs >= 1000) {    // ログのスロットル（毎20msの連発を防ぐ）
98         lastRecoverMs = now;
99         Serial.println("[ACC]_I2C異常検出→_バス復旧を実行");
100     }
101
102     Wire.end();

```

```

103
104 // バスクリア: SCLを9クロック叩いて握られたSDAを解放
105 pinMode(SDA, INPUT_PULLUP);
106 pinMode(SCL, OUTPUT);
107 for (int i = 0; i < 9; i++) {
108     digitalWrite(SCL, LOW); delayMicroseconds(5);
109     digitalWrite(SCL, HIGH); delayMicroseconds(5);
110 }
111 // STOP条件 (SDA: LOW→HIGH while SCL HIGH)
112 pinMode(SDA, OUTPUT);
113 digitalWrite(SDA, LOW); delayMicroseconds(5);
114 digitalWrite(SCL, HIGH); delayMicroseconds(5);
115 digitalWrite(SDA, HIGH); delayMicroseconds(5);
116
117 // Wire再初期化&センサ再設定 (setupADXL345の通信部と同じ)
118 Wire.begin();
119 Wire.beginTransmission(ADXL345_ADDR);
120 Wire.write(REG_POWER_CTL); Wire.write(0x08);
121 Wire.endTransmission();
122 Wire.beginTransmission(ADXL345_ADDR);
123 Wire.write(REG_DATA_FORMAT); Wire.write(0x00);
124 Wire.endTransmission();
125 }
126
127 void AccManager::updateShakedInfo(SyncManager& sync, PotManager& pot) {
128     if (pot.bpmFrag) {
129         sync.sendBPM(pot.getBpmStep());
130         Serial.print("[SHAKE]_BPM送信_step="); Serial.println(pot.getBpmStep());
131         pot.bpmFrag = false;
132     }
133     if (pot.volFrag) {
134         sync.sendVolume(pot.getVolStep());
135         Serial.print("[SHAKE]_VOL送信_step="); Serial.println(pot.getVolStep());
136         pot.volFrag = false;
137     }
138 }

```

B.4 PotManager.h

コード 32 PotManager.h

```

1 // 可変抵抗器
2 #ifndef POT_MANAGER_H
3 #define POT_MANAGER_H
4
5 #include <Arduino.h>
6 #include "SyncManager.h"
7

```

```

8  class PotManager {
9  public:
10     // コンストラクタ（ポート番号などを初期化する）
11     PotManager();
12     //loop内で毎ループ呼ぶ
13     void update();
14     //現在の段階や変化を知るための窓口（ゲッター）
15     uint8_t getBpmStep() const { return currentBpmStep; }
16     uint8_t getVolStep() const { return 6 - currentVolStep; }
17
18     bool bpmFrag = false;
19     bool volFrag = false;
20
21 private:
22     int calcStep(float val);
23     bool bpmUpdatePotentiometers();
24     bool volUpdatePotentiometers();
25
26     // ピン定義
27     const int PIN_BPM = A0; // BPM用可変抵抗器
28     const int PIN_VOL = A1; // 音量用可変抵抗器
29     const unsigned long SAMPLE_INTERVAL = 20; // センサを読みとる周期
30     const int sampleSize = 10; // 移動平均のサンプル数
31     const int STEP_BOUNDARIES[4] = {300, 500, 650, 818};
32
33     // 内部変数
34     unsigned long lastSampleTime = 0; // 前回サンプリング時刻
35     int bpmSamples[10] = {0}; // BPMサンプル配列
36     long bpmTotal = 0; // サンプル合計値
37     float averageBpmVal = 0.0f; // BPM移動平均値
38
39     int volSamples[10] = {0}; // 音量サンプル配列
40     long volTotal = 0; // サンプル合計値
41     float averageVolVal = 0.0f; // 音量移動平均値
42
43     int bpmReadIndex = 0;
44     int volReadIndex = 0;
45     bool bpmBufferFull = false;
46     bool volBufferFull = false;
47
48     uint8_t currentBpmStep = 0; // BPM段階（1～5）
49     uint8_t currentVolStep = 0; // 音量段階（1～5）
50     int prevBpmStep = 0;
51     int prevVolStep = 0;
52
53 };
54
55 #endif

```

B.5 PotManager.cpp

コード 33 PotManager.cpp

```
1 #include "PotManager.h"
2
3 PotManager::PotManager(){
4     // コンストラクタ (初期化処理なし)
5 }
6
7 // ★loopから引っ越してきた時間管理
8 void PotManager::update() {
9     unsigned long now = micros();
10    if (now - lastSampleTime >= SAMPLE_INTERVAL) {
11        lastSampleTime = now;
12        if (bpmUpdatePotentiometers()) bpmFrag = true;
13        if (volUpdatePotentiometers()) volFrag = true;
14    }
15 }
16
17 int PotManager::calcStep(float val) {
18     if (val <= STEP_BOUNDARIES[0]) return 1;
19     else if (val <= STEP_BOUNDARIES[1]) return 2;
20     else if (val <= STEP_BOUNDARIES[2]) return 3;
21     else if (val <= STEP_BOUNDARIES[3]) return 4;
22     else return 5;
23 }
24
25 bool PotManager::bpmUpdatePotentiometers() {
26     bpmTotal -= bpmSamples[bpmReadIndex];
27
28     bpmSamples[bpmReadIndex] = analogRead(PIN_BPM);
29
30     bpmTotal += bpmSamples[bpmReadIndex];
31
32     bpmReadIndex = (bpmReadIndex + 1) % sampleSize;
33
34     if (bpmReadIndex == 0) bpmBufferFull = true;
35
36     if (!bpmBufferFull) return false;
37
38     averageBpmVal = (float)bpmTotal / sampleSize;
39
40     currentBpmStep = calcStep(averageBpmVal);
41
42     if (currentBpmStep != prevBpmStep) {
43         Serial.println("-----_BPM/パラメータ更新_-----");
```

```

44
45     Serial.print("[BPM]_平均センサ値="); Serial.print(averageBpmVal, 1);
46     Serial.print("_段階=");           Serial.print(currentBpmStep);
47
48     prevBpmStep = currentBpmStep;
49     return true;
50 }
51 return false;
52 }
53
54 bool PotManager::volUpdatePotentiometers() {
55     volTotal -= volSamples[volReadIndex];
56
57     volSamples[volReadIndex] = analogRead(PIN_VOL);
58
59     volTotal += volSamples[volReadIndex];
60
61     volReadIndex = (volReadIndex + 1) % sampleSize;
62
63     if (volReadIndex == 0) volBufferFull = true;
64
65     if (!volBufferFull) return false;
66
67     averageVolVal = (float)volTotal / sampleSize;
68
69     currentVolStep = calcStep(averageVolVal);
70
71     if(currentVolStep != prevVolStep){
72         Serial.println("-----_VOLパラメータ更新_-----");
73
74         Serial.print("[VOL]_平均センサ値="); Serial.print(averageVolVal, 1);
75         Serial.print("_段階=");           Serial.print(6 - currentVolStep);
76
77         prevVolStep = currentVolStep;
78         return true;
79     }
80     return false;
81 }

```

B.6 SyncManager.h

コード 34 SyncManager.h

```

1 #ifndef SYNC_MANAGER_H
2 #define SYNC_MANAGER_H
3
4 #include <WiFiS3.h>
5 #include <WiFiUdp.h>

```



```

6
7 //楽器の個別IP (ユニキャスト用)
8 extern IPAddress fluteIP;
9 extern IPAddress pianoIP;
10 extern IPAddress stringsIP;
11 extern IPAddress drumIP;
12
13 //観覧車の個別IP (ユニキャスト用)
14 extern IPAddress ferrisWheelIP;
15
16 //チャンネル (マルチキャスト用)
17 extern IPAddress ferrisWheelGroup; // 観覧車だけが聴くチャンネル (現在未使用)
18 extern IPAddress instrumentGroup; // 楽器全員が聴くチャンネル
19
20 class SyncManager {
21 public:
22     // コンストラクタ (ポート番号などを初期化する)
23     SyncManager();
24     // Wi-FiとUDPを準備する関数
25     void begin(const char ssid[], const char pass[]);
26     // 演奏開始合図('S')を全機へ同時に送信する自作関数
27     void sendStart(uint8_t stepNumber);
28     // BPM情報を送信する関数 (引数の型を追加)
29     void sendBPM(uint8_t stepNumber);
30     // 音量情報を送信する関数 (引数の型を追加)
31     void sendVolume(uint8_t stepNumber);
32     // 演奏終了合図('E')を全機へ同時に送信する関数
33     void sendEnd();
34
35 private:
36     // 複数ターゲットヘラウンドロビンで3回冗長送信を行う共通処理。
37     // ターゲットごとに3連射してから次のターゲットへ進む方式だと、後方の
38     // ターゲットほど到達が遅れて機器間に無視できない時間差が生まれるため、
39     // 1周ごとに全ターゲットへ送ってから次の周に進む方式にしている。
40     void packetRoundRobin(const uint8_t packet[], uint8_t b, const IPAddress targets
41         [], uint8_t targetCount);
42 };
43 #endif

```

B.7 SyncManager.cpp

コード 35 SyncManager.cpp

```

1 #include "SyncManager.h"
2
3 const uint16_t TARGET_PORT = 5005;
4 const uint16_t LOCAL_PORT = 8080;

```

```

5  WiFiUDP Udp;
6
7  IPAddress fluteIP(192, 168, 11, 14);
8  IPAddress pianoIP(192, 168, 11, 12);
9  IPAddress stringsIP(192, 168, 11, 13);
10 IPAddress drumIP(192, 168, 11, 11);
11 IPAddress ferrisWheelIP(192, 168, 11, 10); // 観覧車ユニキャスト
12 IPAddress ferrisWheelGroup(239, 255, 0, 1); // 未使用（マルチキャスト不安定のため）
13 IPAddress instrumentGroup(239, 255, 0, 2);
14
15 SyncManager::SyncManager() {
16     // コンストラクタ（今回は初期化処理なし）（インスタンスが作られた瞬間に、自動的に
17     // 一番最初に1回だけ実行される特殊な関数）
18 }
19 // Wi-FiおよびUDPの初期化フェーズ
20 void SyncManager::begin(const char ssid[], const char pass[]) {
21     WiFi.begin(ssid, pass); // Wi-Fi接続処理を開始
22     // 接続状態が「WL_CONNECTED（接続完了）」になるまでループ待機
23     while (WiFi.status() != WL_CONNECTED) {
24     }
25     Udp.begin(LOCAL_PORT); // 自身のポートを開放し、UDP通信を有効化
26 }
27 // 演奏開始合図（'S'）を全機へラウンドロビンで同時送信
28 void SyncManager::sendStart(uint8_t stepNumber) {
29     // 設計書通りの2バイト固定長ペイロード（第1バイト:ヘッダ'S', 第2バイト:ダミー
30     // 0x00）
31     uint8_t startPacket[2] = {'S', stepNumber};
32     // ドラムはisInstJoinを'S'受信時にリセットするため、他機より早く
33     // 届けたい。ただしラウンドロビン送信なら全ターゲットへの到達は
34     // ほぼ同時になるため、この並び自体はもう重要ではない。
35     IPAddress targets[] = {drumIP, ferrisWheelIP, pianoIP, stringsIP, fluteIP};
36     packetRoundRobin(startPacket, 2, targets, 5);
37 }
38 // BPM情報の送信（引数で段階番号を受け取る）
39 // 'B' は観覧車のみが受信するため、観覧車へユニキャスト送信
40 void SyncManager::sendBPM(uint8_t stepNumber){
41     uint8_t bpmPacket[2] = {'B', stepNumber};
42     IPAddress targets[] = {ferrisWheelIP};
43     packetRoundRobin(bpmPacket, 2, targets, 1);
44 }
45 // 音量情報の送信（引数で段階番号を受け取る）
46 // 'V' は全楽器機が受信するため、各楽器機へユニキャスト送信
47 void SyncManager::sendVolume(uint8_t stepNumber){
48     uint8_t volumePacket[2] = {'V', (uint8_t)(6 - stepNumber)};
49     IPAddress targets[] = {fluteIP, pianoIP, stringsIP, drumIP};
50     packetRoundRobin(volumePacket, 2, targets, 4);
51 }
52 // 演奏終了合図（'E'）を全機へラウンドロビンで同時送信

```

```

51 void SyncManager::sendEnd(){
52     uint8_t endPacket[2] = {'E', 0x00};
53     IPAddress targets[] = {drumIP, ferrisWheelIP, pianoIP, stringsIP, fluteIP};
54     packetRoundRobin(endPacket, 2, targets, 5);
55 }
56
57 // 複数ターゲットへのラウンドロビン3回冗長送信（パケットロス対策）
58 // 1周目で全ターゲットに1回ずつ送り、20ms空けてから2周目、3周目と繰り返す。
59 // ターゲットごとに3連射してから次へ進む方式だと、後方のターゲットほど
60 // 到達が遅れて機器間で無視できない時間差が生まれるため、この方式にしている。
61 void SyncManager::packetRoundRobin(const uint8_t packet[], uint8_t b, const IPAddress
    targets[], uint8_t targetCount){
62     for (int round = 0; round < 3; round++) {
63         for (uint8_t t = 0; t < targetCount; t++) {
64             Udp.beginPacket(targets[t], TARGET_PORT); // 送信先IPとポートをセット
65             Udp.write(packet, b);
66             Udp.endPacket(); // パケットをパースして実際に送信
67         }
68         // 最後の周（3周目）以外は、20msのインターバル（遅延）を挟む
69         if (round < 2) {
70             delay(20);
71         }
72     }
73 }

```

C 中継機デバイス（Display）のプログラムソース

本節では、中継機デバイスのモータ回転制御およびLEDマトリクス表示を行う制御プログラムの全文を示す。

C.1 FerrisWheel.ino

コード 36 FerrisWheel.ino

```

1  #include "MotorControl.h"
2  #include "Display.h"
3  #include <WiFiS3.h>
4  #include <WiFiUdp.h>
5
6  const char SSID[] = "hackathon001-WPA2";
7  const char PASS[] = "hackathon001";
8
9  IPAddress localIP(192, 168, 11, 10);
10 IPAddress gateway(192, 168, 11, 1);
11 IPAddress subnet(255, 255, 255, 0);
12
13 const uint16_t LOCAL_PORT = 5005;

```

```

14
15 MotorControl motor;
16
17 WiFiUDP Udp;
18 uint8_t packetBuffer[10];
19
20 // 演奏中フラグ。'S'でtrue / 'E'でfalse。
21 // 'B'(BPM変更)は演奏中のみ受け付け、開始前の不意の'B'でモーターが回り出すのを防ぐ。
22 bool running = false;
23
24 void setup() {
25     Serial.begin(115200);
26     delay(2000);
27
28     Serial.println("---_FerrisWheel_Setup_Start_---");
29
30     motor.begin();
31     motor.createRpmTable();
32
33     WiFi.config(localIP, gateway, subnet);
34     WiFi.begin(SSID, PASS);
35     Serial.print("Connecting_to_WiFi...");
36     while (WiFi.status() != WL_CONNECTED) {
37         delay(500);
38         Serial.print(".");
39     }
40     Serial.println("\nWiFi_Connected!");
41     Serial.print("IP:_"); Serial.println(WiFi.localIP());
42
43     Udp.begin(LOCAL_PORT);
44     Serial.print("Listening_on_Unicast_Port_"); Serial.println(LOCAL_PORT);
45
46     displaySetup();
47 }
48
49 void loop() {
50     uint8_t packetSize = Udp.parsePacket();
51     if (packetSize <= 0) return;
52
53     if (packetSize != 2) {
54         Serial.print("想定外のパケットサイズ:_"); Serial.println(packetSize);
55         while (Udp.available()) Udp.read();
56         return;
57     }
58
59     Udp.read(packetBuffer, sizeof(packetBuffer));
60     char header = packetBuffer[0];
61     uint8_t payload = packetBuffer[1];

```

```

62
63     switch (header) {
64         case 'S':
65             Serial.print("演奏開始_('S')_payload="); Serial.println(payload);
66             // payload に BPM 段階が入っているのでそれで起動（元コードの global 変数
              参照バグを修正）
67             running = true;
68             motor.startRamp(payload);
69             displayBPM(payload);
70             // 冗長送信の残りパケットを捨てる
71             while (Udp.parsePacket() > 0) {
72                 while (Udp.available()) Udp.read();
73             }
74             break;
75
76         case 'B':
77             if (!running) {
78                 Serial.println("演奏前の_'B'_を無視");
79                 break;
80             }
81             Serial.print("BPM変更_('B')_段階="); Serial.println(payload);
82             motor.changeBPM(payload);
83             displayBPM(payload);
84             break;
85
86         case 'E':
87             Serial.println("演奏終了_('E')");
88             running = false;
89             motor.stopRamp();
90             clearDisplay();
91             break;
92
93         default:
94             Serial.print("想定外のヘッダ:_"); Serial.println(header);
95             break;
96     }
97 }

```

C.2 Display.h

コード 37 Display.h

```

1  #ifndef DISPLAY_H
2  #define DISPLAY_H
3
4  #include <Arduino.h>
5  #include "Arduino_LED_Matrix.h"
6

```

```

7  #define BPM_STEP_1  60
8  #define BPM_STEP_2  90
9  #define BPM_STEP_3  120
10 #define BPM_STEP_4  150
11 #define BPM_STEP_5  180
12
13 #define FONT_WIDTH   3
14 #define FONT_HEIGHT  5
15 #define MATRIX_ROWS  8
16 #define MATRIX_COLS  12
17
18 void displaySetup();
19 void displayBPM(uint8_t stepNumber);
20 void clearDisplay();
21 void drawDigit(int digit, int x_offset);
22
23 extern const uint8_t font3x5[10][15];
24 extern ArduinoLEDMatrix matrix;
25
26 #endif

```

C.3 Display.cpp

コード 38 Display.cpp

```

1  #include "Display.h"
2  #include "MotorControl.h"
3  #include "Arduino_LED_Matrix.h"
4
5  const uint8_t font3x5[10][15] = {
6      // 0
7      {1,1,1, 1,0,1, 1,0,1, 1,0,1, 1,1,1},
8      // 1
9      {0,1,0, 1,1,0, 0,1,0, 0,1,0, 1,1,1},
10     // 2
11     {1,1,1, 0,0,1, 1,1,1, 1,0,0, 1,1,1},
12     // 3
13     {1,1,1, 0,0,1, 0,1,1, 0,0,1, 1,1,1},
14     // 4
15     {1,0,1, 1,0,1, 1,1,1, 0,0,1, 0,0,1},
16     // 5
17     {1,1,1, 1,0,0, 1,1,1, 0,0,1, 1,1,1},
18     // 6
19     {1,1,1, 1,0,0, 1,1,1, 1,0,1, 1,1,1},
20     // 7
21     {1,1,1, 0,0,1, 0,0,1, 0,0,1, 0,0,1},
22     // 8
23     {1,1,1, 1,0,1, 1,1,1, 1,0,1, 1,1,1},

```

```

24 // 9
25 {1,1,1, 1,0,1, 1,1,1, 0,0,1, 1,1,1}
26 };
27
28 static const int bpmTable[5] = {
29     BPM_STEP_1, BPM_STEP_2, BPM_STEP_3, BPM_STEP_4, BPM_STEP_5
30 };
31
32 ArduinoLEDMatrix matrix;
33
34 void displaySetup() {
35     matrix.begin();
36     uint32_t warmUp[3] = {0, 0, 0};
37     matrix.loadFrame(warmUp);
38     delay(200);
39 }
40
41 static uint8_t frameBuffer[MATRIX_ROWS][MATRIX_COLS];
42 static int currentBPM = -1;
43
44 void displayBPM(uint8_t stepNumber) {
45     if (stepNumber < 1 || stepNumber > 5) return;
46     uint8_t bpm = bpmTable[stepNumber - 1];
47
48     memset(frameBuffer, 0, sizeof(frameBuffer));
49
50     int digits[3];
51     int numDigits;
52     if (bpm >= 100) {
53         numDigits = 3;
54         digits[0] = bpm / 100;
55         digits[1] = (bpm / 10) % 10;
56         digits[2] = bpm % 10;
57     } else if (bpm >= 10) {
58         numDigits = 2;
59         digits[0] = bpm / 10;
60         digits[1] = bpm % 10;
61     } else {
62         numDigits = 1;
63         digits[0] = bpm;
64     }
65
66     int totalWidth = numDigits * FONT_WIDTH + (numDigits - 1) * 1;
67     int startX = (MATRIX_COLS - totalWidth) / 2;
68
69     for (int i = 0; i < numDigits; i++) {
70         drawDigit(digits[i], startX + i * (FONT_WIDTH + 1));
71     }

```

```

72
73     uint32_t bitmap[3] = {0, 0, 0};
74     for (int row = 0; row < MATRIX_ROWS; row++) {
75         for (int col = 0; col < MATRIX_COLS; col++) {
76             if (frameBuffer[row][col]) {
77                 int n = row * MATRIX_COLS + col;
78                 bitmap[n / 32] |= (1UL << (31 - (n % 32)));
79             }
80         }
81     }
82     matrix.loadFrame(bitmap);
83 }
84
85 void drawDigit(int digit, int x_offset) {
86     if (digit < 0 || digit > 9) return;
87     const int y_offset = (MATRIX_ROWS - FONT_HEIGHT) / 2;
88     for (int y = 0; y < FONT_HEIGHT; y++) {
89         for (int x = 0; x < FONT_WIDTH; x++) {
90             int col = x_offset + x;
91             int row = y_offset + y;
92             if (col >= 0 && col < MATRIX_COLS && row >= 0 && row < MATRIX_ROWS) {
93                 frameBuffer[row][col] = font3x5[digit][y * FONT_WIDTH + x];
94             }
95         }
96     }
97 }
98
99 void clearDisplay() {
100     currentBPM = -1;
101     uint32_t blank[3] = {0, 0, 0};
102     matrix.loadFrame(blank);
103 }

```

C.4 MotorControl.h

コード 39 MotorControl.h

```

1  #ifndef MOTOR_CONTROL_H
2  #define MOTOR_CONTROL_H
3
4  #include <Arduino.h>
5
6  class MotorControl {
7  public:
8      MotorControl();
9      void begin();
10     void createRpmTable();
11     void startRamp(uint8_t stepNumber);

```



```

12     void changeBPM(uint8_t stepNumber);
13     void stopRamp();
14
15 private:
16     void stepToNumber(uint8_t stepNumber);
17     void updateStatus();
18
19     const uint8_t PIN_MOTOR_PWM = 5;
20     const uint8_t bpmTable[5] = {60, 90, 120, 150, 180};
21     const float VCC = 5.0;
22     const uint8_t pwmTable[5] = {26, 28, 30, 32, 37}; // 仮の数値
23
24     uint8_t currentBPM = 0;
25     float RPM = 0;
26     float omega = 0;
27     uint8_t pwmValue = 0;
28     float V_average = 0;
29     float rpmTable[5] = {};
30 };
31
32 #endif

```

C.5 MotorControl.cpp

コード 40 MotorControl.cpp

```

1  #include "MotorControl.h"
2
3  MotorControl::MotorControl() {}
4
5  void MotorControl::begin() {
6      pinMode(PIN_MOTOR_PWM, OUTPUT);
7      analogWrite(PIN_MOTOR_PWM, 0);
8  }
9
10 void MotorControl::createRpmTable() {
11     for (uint8_t i = 0; i < 5; i++) {
12         rpmTable[i] = bpmTable[i] / 16.0f;
13     }
14     Serial.println("[Motor]_rpmTable_初期化完了");
15 }
16
17 void MotorControl::startRamp(uint8_t stepNumber) {
18     stepToNumber(stepNumber);
19     Serial.println("[Motor]_Start_Ramp_Up...");
20     for (int i = 0; i <= pwmValue; i += 5) {
21         analogWrite(PIN_MOTOR_PWM, i);
22         delay(40);

```

```

23     }
24     analogWrite(PIN_MOTOR_PWM, pwmValue);
25     Serial.println("[Motor]_Target_speed_reached.");
26 }
27
28 void MotorControl::stopRamp() {
29     Serial.println("[Motor]_Start_Ramp_Down...");
30     for (int i = pwmValue; i >= 0; i -= 5) {
31         analogWrite(PIN_MOTOR_PWM, i);
32         delay(40);
33     }
34     analogWrite(PIN_MOTOR_PWM, 0);
35     Serial.println("[Motor]_Stopped.");
36 }
37
38 void MotorControl::changeBPM(uint8_t stepNumber) {
39     stepToNumber(stepNumber);
40     updateStatus();
41     analogWrite(PIN_MOTOR_PWM, pwmValue);
42     Serial.print("[Motor]_Speed_Changed._PWM:_"); Serial.println(pwmValue);
43 }
44
45 void MotorControl::stepToNumber(uint8_t stepNumber) {
46     if (stepNumber < 1 || stepNumber > 5) return;
47     currentBPM = bpmTable[stepNumber - 1];
48     pwmValue    = pwmTable[stepNumber - 1];
49     RPM         = rpmTable[stepNumber - 1];
50 }
51
52 void MotorControl::updateStatus() {
53     omega      = 2.0 * PI * RPM / 60.0f;
54     V_average = VCC * (pwmValue / 256.0f);
55 }

```

D 回転速度実測プログラム (LaserIntervalTest) のプログラムソース

本節では、中継機デバイスの回転速度実測に用いた LaserIntervalTest.ino の全文を示す。

D.1 LaserIntervalTest.ino

コード 41 LaserIntervalTest.ino

```

1 // CdSセルでのレーザー検知間隔を計測するスケッチ
2 // 連続する2回のパルス検知の時間差(interval)をシリアルモニタに表示し続ける。
3 //
4 // 検知ロジックは HCK_inst_0701 の drum_0701.ino / notDrum_0701.ino で使っている
5 // SyncManager::detectLight() をそのまま流用 (起動時キャリブレーション+

```

```

6 // 直近ピーク履歴による適応しきい値）。BPM段階への変換は行わず、生のinterval[ms]
7 // だけを出す。
8
9 const int CDS_PIN = A0;
10
11 const int CDS_ON_THRESHOLD = 800; // 起動直後だけ使う仮しきい値
12 const int CDS_OFF_THRESHOLD = 780; // 起動直後だけ使う仮しきい値
13 const int CDS_MIN_DELTA = 25; // 暗い状態との差がこれ未満ならレーザー扱いしない
14 const uint8_t CDS_PEAK_HISTORY = 4; // 直近何回分のピークからしきい値を決めるか
15 const uint8_t CDS_ON_RATIO = 65; // baselineからピークまでの65%をONしきい値にする
16 const uint8_t CDS_OFF_RATIO = 35; // baselineからピークまでの35%をOFFしきい値にする
17 const int CDS_THRESHOLD_STEP_LIMIT = 25; // 1回の検出でしきい値が動きすぎない上限
18 const unsigned long CDS_STARTUP_CALIBRATION_MS = 1000; // 起動直後の誤検出防止時間
19 const unsigned long DEBOUNCE_MS = 200;
20
21 class LaserDetector {
22 public:
23     LaserDetector()
24         : lastDetectTime(0), laserOn(false),
25           baseline(0), baselineReady(false),
26           onThreshold(CDS_ON_THRESHOLD), offThreshold(CDS_OFF_THRESHOLD),
27           pulsePeak(0), peakIndex(0), peakCount(0),
28           calibrated(false), startupCalibrated(false), waitingForRelease(false),
29           calibrationStartMs(0), startupMin(1023), releaseThreshold(0) {
30         clearPeakHistory();
31     }
32
33     // 新しいパルスを検知したら true を返す。interval[ms] を out に書く（前回検知が無ければ0）。
34     bool update(int cdsValue, unsigned long nowMs, unsigned long &intervalOut) {
35         if (calibrationStartMs == 0) calibrationStartMs = nowMs;
36         if (!detectLight(cdsValue, nowMs)) return false;
37         if (nowMs - lastDetectTime < DEBOUNCE_MS) return false;
38
39         intervalOut = (lastDetectTime != 0) ? (nowMs - lastDetectTime) : 0;
40         bool hasInterval = (lastDetectTime != 0);
41         lastDetectTime = nowMs;
42         return hasInterval;
43     }
44
45 private:
46     unsigned long lastDetectTime;
47     bool laserOn;
48     int baseline;
49     bool baselineReady;
50     int onThreshold;
51     int offThreshold;
52     int pulsePeak;

```

```

53     int                peakHistory[CDS_PEAK_HISTORY];
54     uint8_t            peakIndex;
55     uint8_t            peakCount;
56     bool               calibrated;
57     bool               startupCalibrated;
58     bool               waitingForRelease;
59     unsigned long      calibrationStartMs;
60     int                startupMin;
61     int                releaseThreshold;
62
63     bool detectLight(int v, unsigned long nowMs) {
64         if (!baselineReady) {
65             baseline = v;
66             startupMin = v;
67             baselineReady = true;
68             updateFallbackThresholds();
69         }
70
71         // 起動直後にレーザーが当たっていても、最初の1秒は検出せず暗い側の値を探す
72         if (!startupCalibrated) {
73             if (v < startupMin) startupMin = v;
74             baseline = startupMin;
75             updateFallbackThresholds();
76             if (nowMs - calibrationStartMs < CDS_STARTUP_CALIBRATION_MS) return false;
77             startupCalibrated = true;
78             waitingForRelease = (v >= onThreshold);
79             releaseThreshold = (baseline > 1023 - CDS_MIN_DELTA) ? baseline - CDS_MIN_DELTA
                : offThreshold;
80             return false;
81         }
82
83         if (waitingForRelease) {
84             if (v <= releaseThreshold) {
85                 baseline = v;
86                 updateFallbackThresholds();
87                 waitingForRelease = false;
88             }
89             return false;
90         }
91
92         if (!laserOn) {
93             if (v < baseline) baseline = v;
94             else baseline = (baseline * 31 + v) / 32;
95             if (!calibrated) updateFallbackThresholds();
96         }
97
98         if (!laserOn && v >= onThreshold) {
99             laserOn = true;

```

```

100     pulsePeak = v;
101     return true;
102 }
103
104 if (laserOn) {
105     if (v > pulsePeak) pulsePeak = v;
106     if (v <= offThreshold) {
107         laserOn = false;
108         registerPeak(pulsePeak);
109     }
110 }
111 return false;
112 }
113
114 void registerPeak(int peak) {
115     if (peak - baseline < CDS_MIN_DELTA) return;
116
117     peakHistory[peakIndex] = peak;
118     peakIndex = (peakIndex + 1) % CDS_PEAK_HISTORY;
119     if (peakCount < CDS_PEAK_HISTORY) peakCount++;
120     if (peakCount < 3) return;
121
122     long sum = 0;
123     for (uint8_t i = 0; i < peakCount; i++) sum += peakHistory[i];
124     int peakAvg = sum / peakCount;
125     int amplitude = peakAvg - baseline;
126     if (amplitude < CDS_MIN_DELTA) return;
127
128     int targetOn = constrain(baseline + (amplitude * CDS_ON_RATIO) / 100, 0, 1023);
129     int targetOff = constrain(baseline + (amplitude * CDS_OFF_RATIO) / 100, 0,
        targetOn - 5);
130
131     if (!calibrated) {
132         onThreshold = targetOn;
133         offThreshold = targetOff;
134         calibrated = true;
135     } else {
136         onThreshold = limitStep(onThreshold, targetOn, CDS_THRESHOLD_STEP_LIMIT);
137         offThreshold = limitStep(offThreshold, targetOff, CDS_THRESHOLD_STEP_LIMIT);
138     }
139     if (offThreshold >= onThreshold) offThreshold = max(0, onThreshold - 5);
140 }
141
142 void updateFallbackThresholds() {
143     onThreshold = constrain(baseline + CDS_MIN_DELTA, 0, 1023);
144     offThreshold = constrain(baseline + CDS_MIN_DELTA / 2, 0, max(0, onThreshold - 5));
145 }

```

```

145     }
146
147     int limitStep(int current, int target, int limit) {
148         if (target > current + limit) return current + limit;
149         if (target < current - limit) return current - limit;
150         return target;
151     }
152
153     void clearPeakHistory() {
154         for (uint8_t i = 0; i < CDS_PEAK_HISTORY; i++) peakHistory[i] = 0;
155     }
156 };
157
158 LaserDetector detector;
159
160 void setup() {
161     Serial.begin(115200);
162     Serial.println("---レーザー間隔計測_Setup_Started_---");
163 }
164
165 void loop() {
166     unsigned long now = millis();
167     int cdsValue = analogRead(CDS_PIN);
168     unsigned long interval = 0;
169
170     if (detector.update(cdsValue, now, interval)) {
171         Serial.print("[interval]_");
172         Serial.print(interval);
173         Serial.println("_ms");
174     }
175 }

```